

EE430 Term Project – Part 2

Bera Alvan 2442416

Onur Arıkan 2442440

Introduction

This project includes the implementation of a dual-tone multi-frequency (DTMF) signal transmitter and receiver app using MATLAB App Designer. The app enables the user to generate DTMF signals using a telephone-like four-by-four keypad. The receiver part uses the microphone to detect and decode DTMF signals.

Graphical User Interface

The GUI follows a minimalistic design approach.

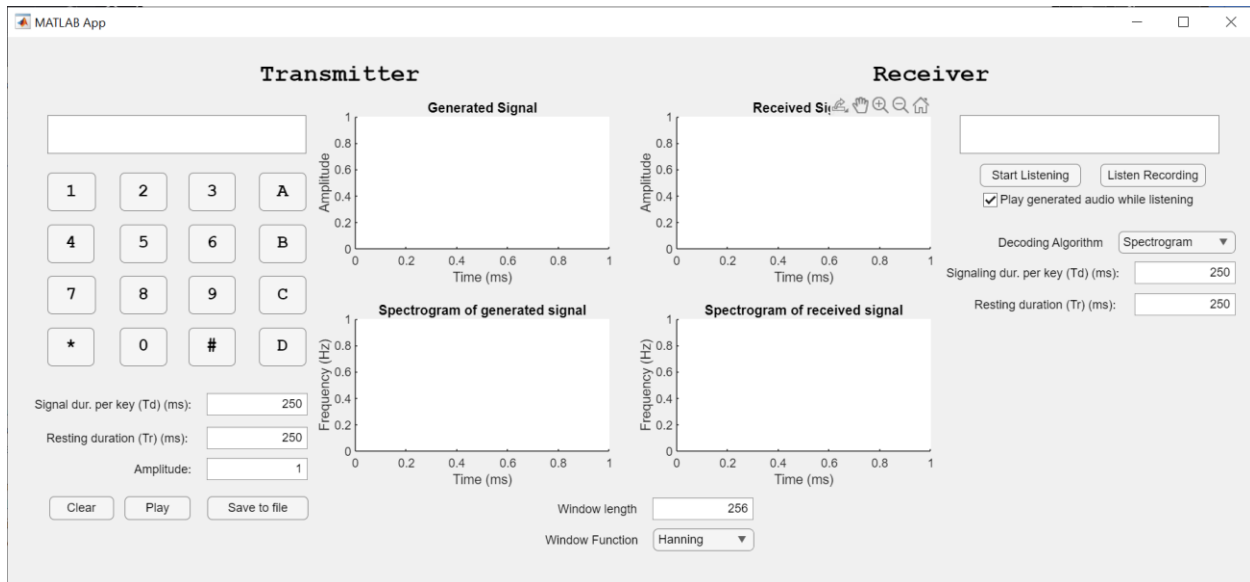


Figure 1: The graphical user interface

Using the Transmitter

The transmitter features a keypad using which the users can enter a character sequence. The keypad includes the standard telephone keys that range from 1 to 9 and A to D. App will generate a DTMF signal each time a key is pressed and append it to the previous keys that are pressed. The pressed keys are shown in the white “screen”.

App will automatically generate a time-domain plot and a spectrogram of the DTMF signal each time a key is pressed.

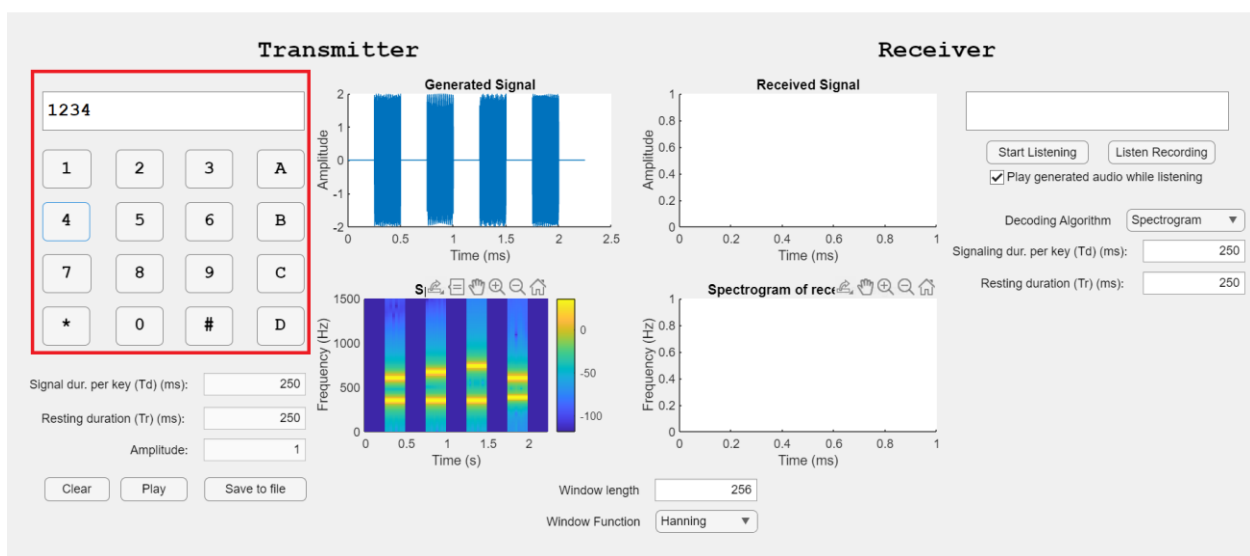


Figure 2: The keypad

It is also possible to set the desired pulse, resting duration and amplitude for the signal creation. However, this is only possible before a key is pressed. Once a signal is created, it is not possible to change these variables.

To clear the generated signal, use the “Clear” button. This will reset the plot and the spectrogram as well.

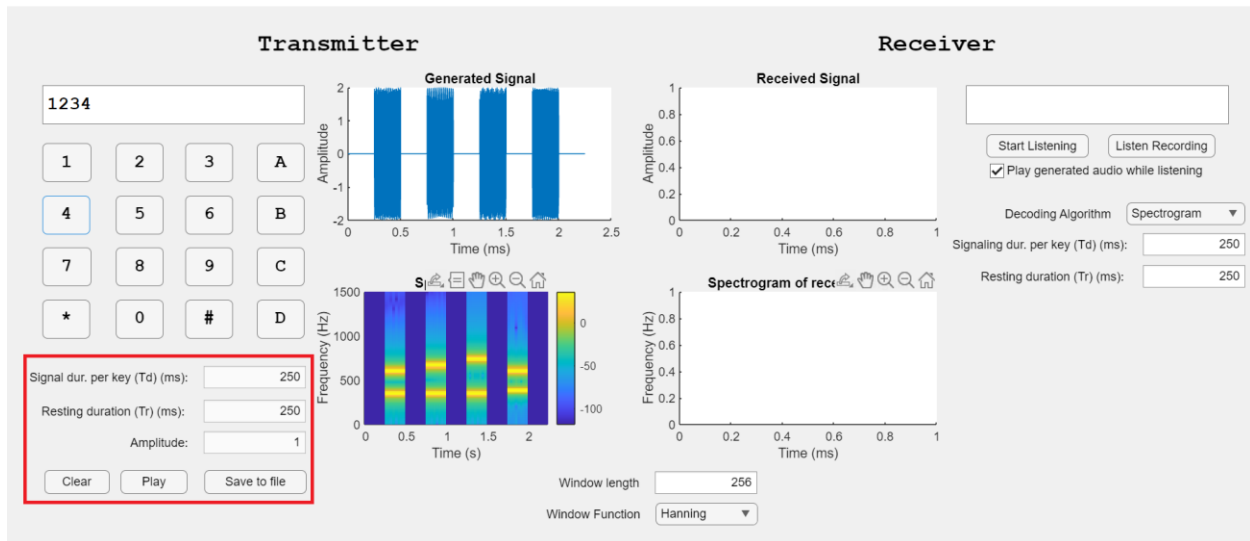


Figure 3: A DTMF signal "1234" is created with $T_d=250\text{ms}$ and $T_r=250\text{ms}$

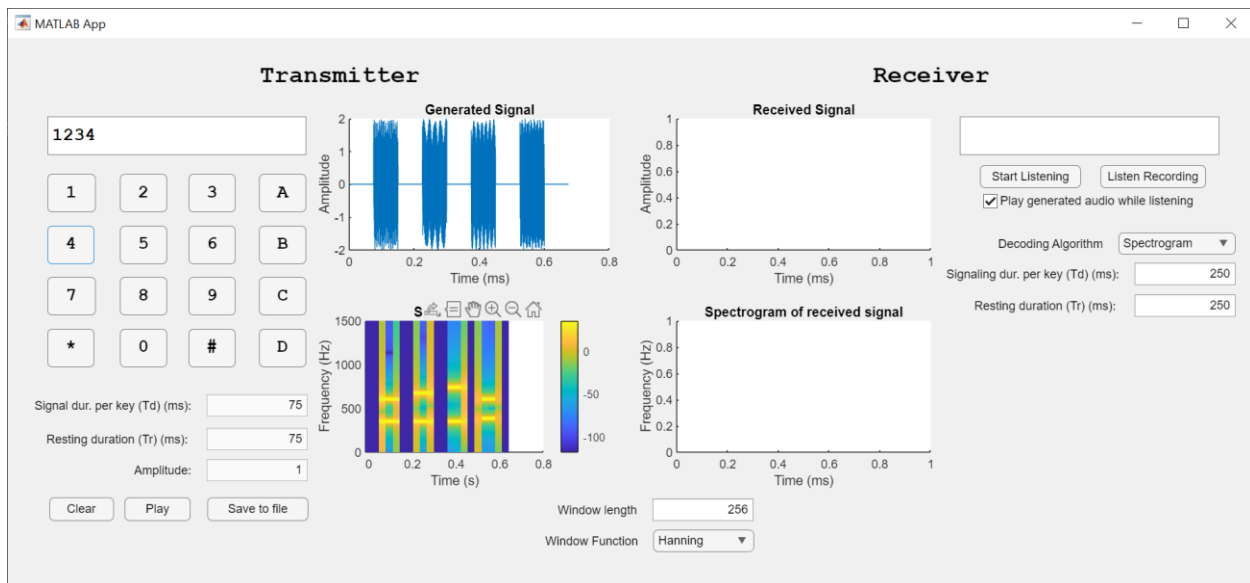


Figure 4: Generating the same sequence "1234" but with a 150ms pulse length and delay

The generated DTMF signal can be saved as a .wav file using the “Save to file” button.

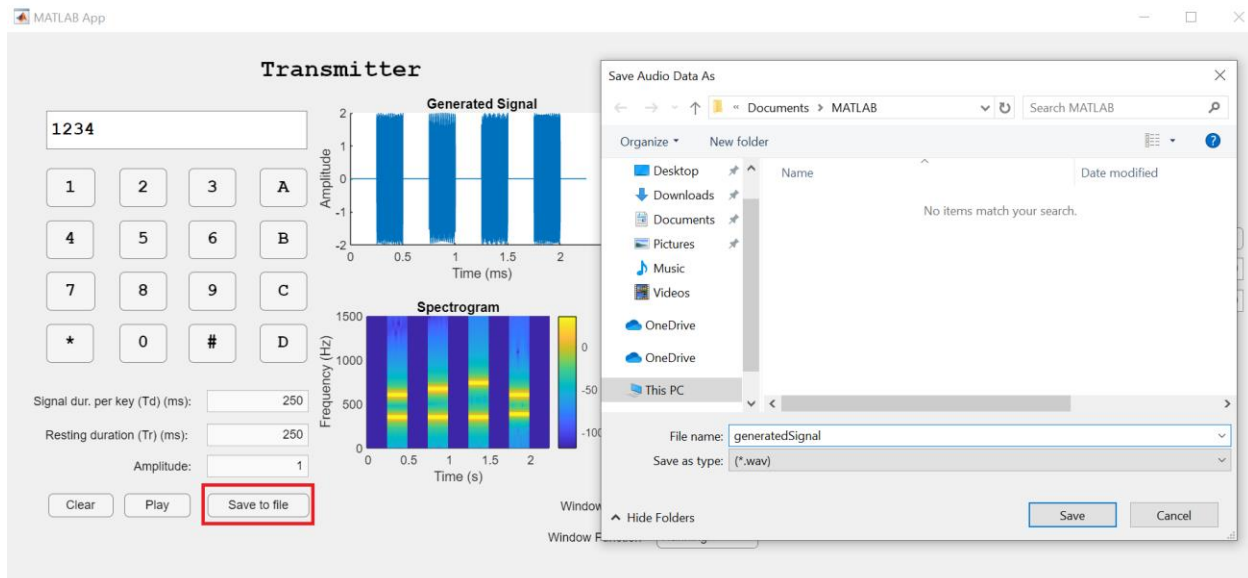


Figure 5: Saving the created DTMF signal

The spectrogram parameters such as window type and length can be easily modified using the inputs below the spectrograms. These parameters apply to both the transmitter and the receiver.

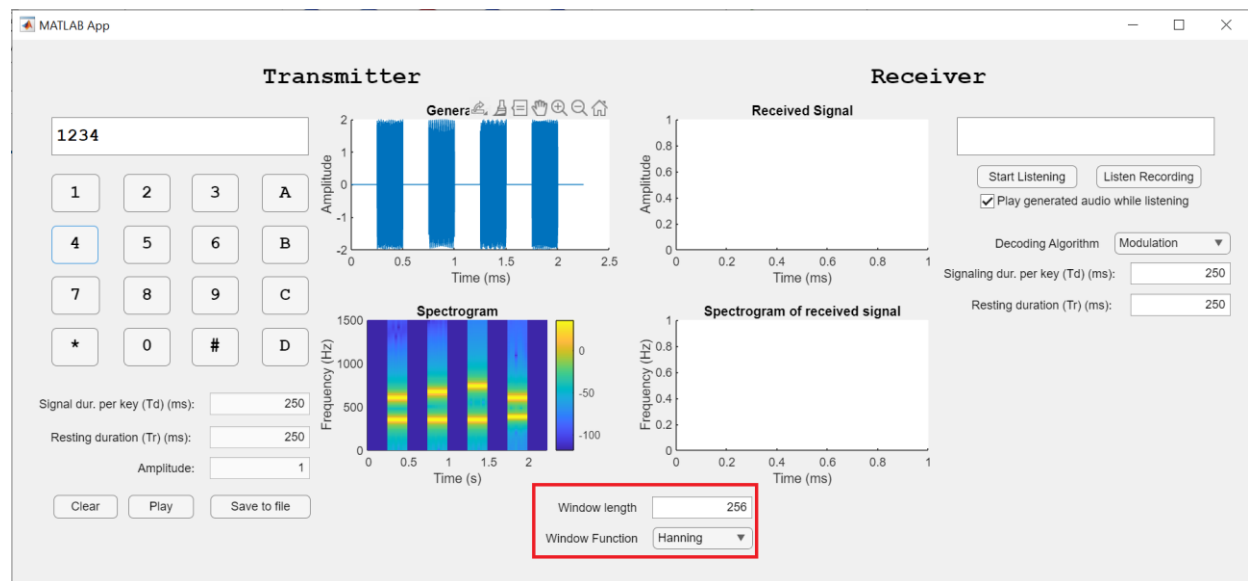


Figure 6: Spectrogram parameter selection

Using the Receiver

The receiver part allows decoding DTMF signals using a microphone.

To start listening, press “Start Listening” button. Once pressed, the button will change to “Stop Listening”.

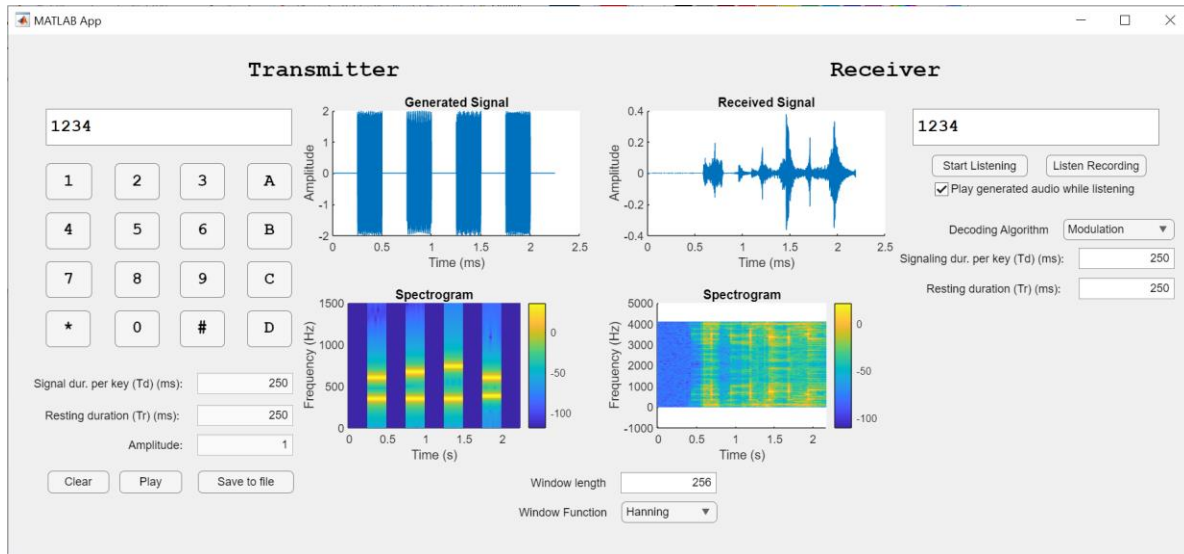


Figure 7: The receiver part can be seen at the right

To stop listening, use “Stop Listening” button. When the recording is complete, a time-domain plot and a spectrogram will be automatically generated.

Checking the “Play generated audio while listening” checkbox will automatically play the generated DTMF signal in the transmitter side as the recording progresses. This can be unchecked to provide signals from a different source.

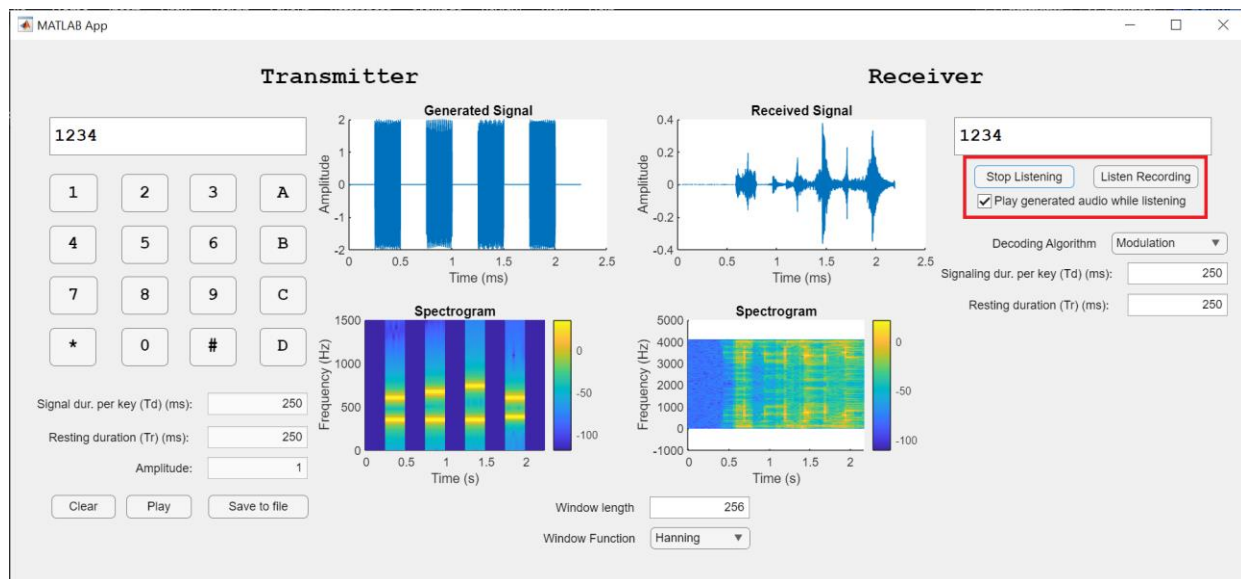


Figure 8: Buttons for listening

When the recording is stopped, the app will automatically decode the listened signal using Modulation and display the result in the white “screen” above.

After the recording is stopped, it is possible to re-decode the listened signal using any of the available algorithms by simply selecting the desired one from the drop-down menu. Once another algorithm is selected, the app will automatically decode the signal again. Three decoding algorithms are available, one being modulation-based, another uses a filter bank, and the other is a spectrogram-based approach.

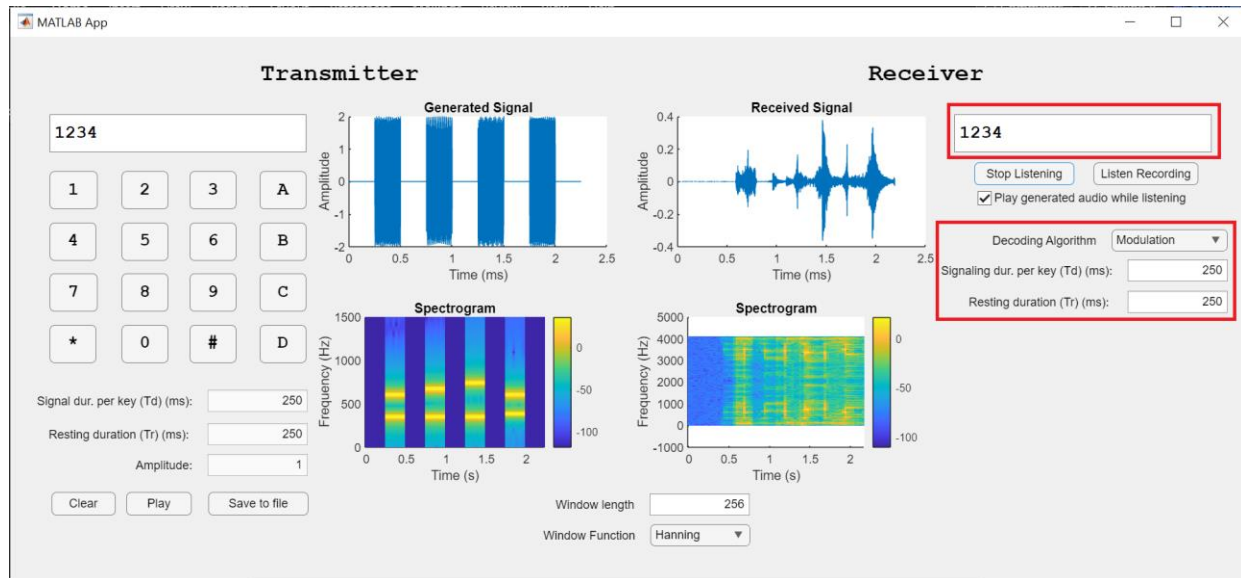


Figure 9: Decoding of the listened signal

Decoding by Modulation

This method relies on the fact that modulating a tone by a sine wave will generate components of both sum and the difference of the frequencies of the original tone and the sine wave. This will effectively shift the frequency range. When the input contains the same frequency as the modulation frequency, a DC component will be created. This is useful, because we can add a filter after the modulation to make the system a controllable bandpass filter. Thus, we do not need to use 16 bandpass filters as in the filter bank approach.

Once a DTMF signal is deemed to be present, the input is sliced to leave each of the tones on its own. For each tone, the controlled filter should be set to different frequencies and applied to the tones one after another. Because of this, modulation method is slower than using a filter bank, which applies the filters in parallel. Modulation decoding would be unfavorable if we decoded our signal in real-time as it is slower.

Decoding Using a Filter Bank

This approach is simpler than the others. We constructed 16 Butterworth bandpass filters for each of the DTMF high and low frequencies with a passband of 25Hz around the center frequency. The recorded audio file is filtered by each filter and the outputs are put into a matrix. In an active pulse, we compare the filter outputs to find the two highest frequency components. Then, we compare these two frequencies with the DTMF frequency table and determine the key they represent. This approach is much faster than the others, as it processes different frequencies in parallel. Decoding using a filter bank is the better choice for real-time applications because of its speed.

Decoding Using a Spectrogram

The process initiates with the generation of a spectrogram from the DTMF signal. A spectrogram is a 2D representation that illustrates the frequency content of a signal over time. The spectrogram used is the one we wrote in Part 1 of this project. The columns of spectrogram matrix created are fft of x signals selected segment. These segments are determined by a window and each segment's fft creates a column for spectrogram. Time bins in the spectrogram are essentially time intervals, and the colors or intensity in the spectrogram represent the magnitude of the frequency components at different time points.

Given the known pulse duration, duration between pulses, and the sampling rate, the spectrogram is divided into time bins that correspond to each DTMF signal. This step involves aligning the spectrogram with the expected timing of DTMF pulses.

Peaks in the spectrogram correspond to significant frequency components at specific time intervals. In the context of DTMF decoding, these peaks represent the presence of individual tones. Advanced signal processing techniques are employed to detect these peaks accurately.

Following peak detection, the two highest peaks within the identified time bins are selected. These two peaks correspond to the dominant frequencies present during a particular DTMF

pulse. The choice of two peaks aligns with the dual-frequency nature of DTMF signals, where each keypress is characterized by a combination of high and low frequencies.

The selected frequencies are then matched to the DTMF frequency table. By comparing the identified frequencies with the known frequency pairs associated with each DTMF key, the decoding system can determine the specific key pressed during the corresponding time interval.

The Comparison of Three Methods Used for Decoding

Spectrogram-Based Decoding:

- Utilizes time-frequency representation.
- Visualizes frequency content over time.
- Involves identifying peaks in the spectrogram.
- Requires advanced signal processing for spectrogram generation and peak detection.
- Adaptable but may involve complex algorithms.
- Adaptable to variations in signal characteristics.
- Can handle changes in amplitude and duration of DTMF pulses.
- More susceptible to noise

Filter Bank Approach:

- Employs a set of bandpass filters.
- Isolates specific frequency bands associated with DTMF tones.
- Involves the construction and utilization of bandpass filters.
- Simpler in terms of filter design but may require precise tuning, efficient for known, standard DTMF signals.
- Limited adaptability to variations in signal characteristics.
- Parallel processing enhances speed but may require more resources.
- Commonly used in real-time DTMF decoding applications.

Modulation-Based Decoding:

- Shifts frequency components through modulation.
- Utilizes modulation and controlled filtering.
- Sequential processing may introduce some complexity.
- Adaptability may be influenced by modulation parameters.
- Sequential processing may be slower but offers controlled analysis.
- Sequential processing may introduce delays, affecting real-time applications.

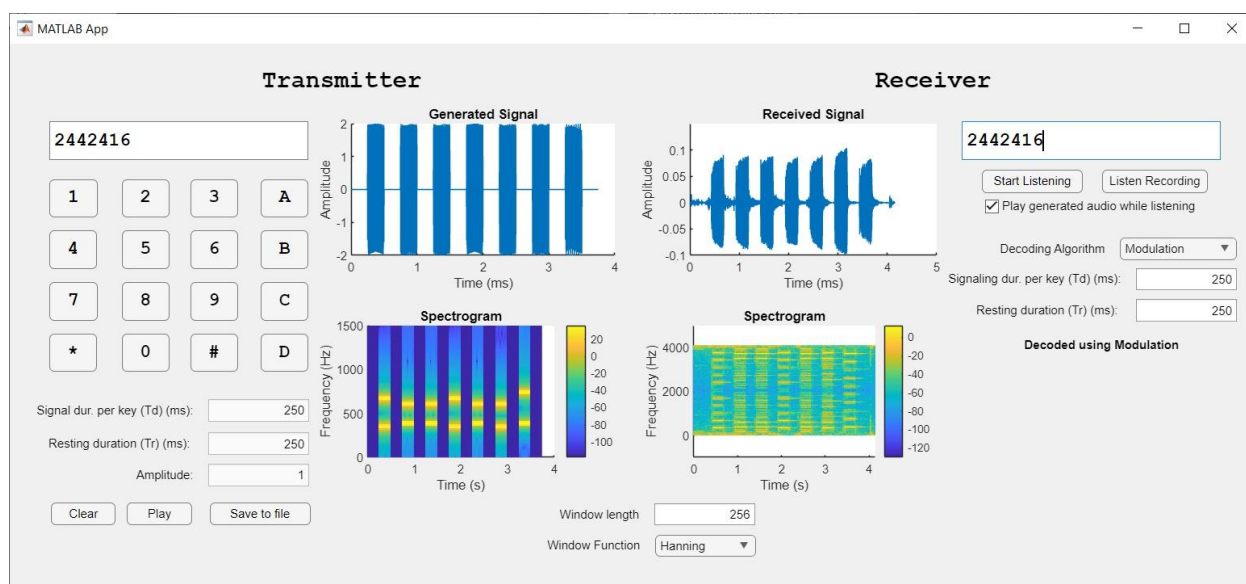
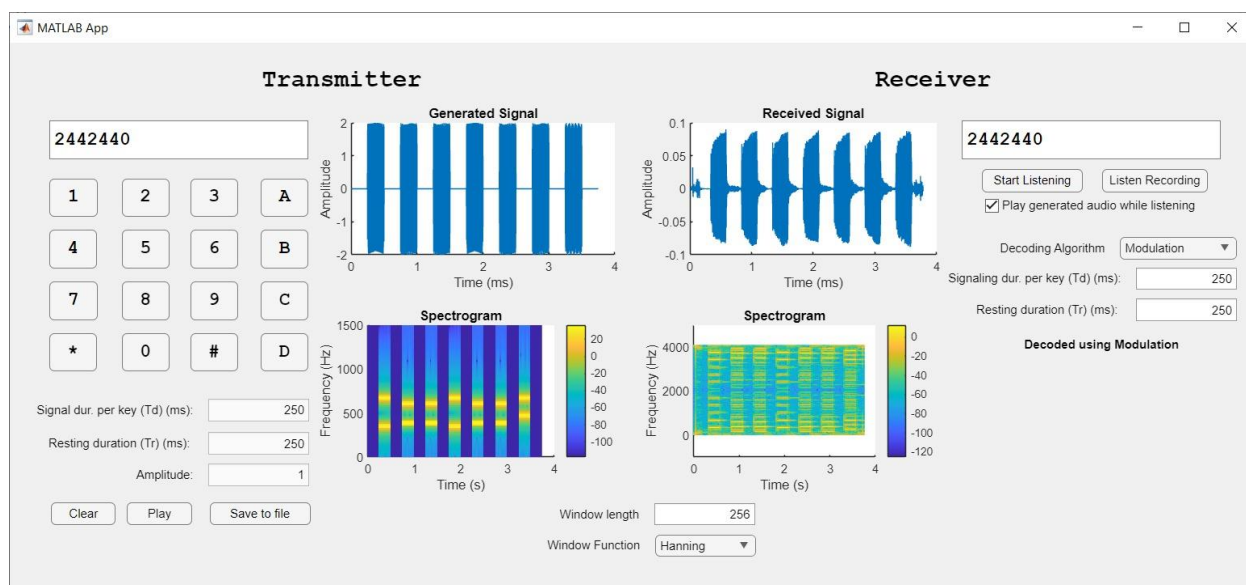
Conclusion:

The choice of DTMF decoding method depends on the specific requirements of the application. Spectrogram-based decoding offers visual insights and adaptability, making it suitable for diverse signal variations. Filter bank approaches provide a balance between simplicity and efficiency, often favored in real-time applications. Modulation-based decoding, while offering flexibility, may be slightly slower due to sequential processing. Ultimately, the most appropriate method depends on factors such as the complexity of the signal, computational resources, and real-time processing requirements. For our case, the best results were taken with filter bank decoding, in demo day. The results are given after this part.

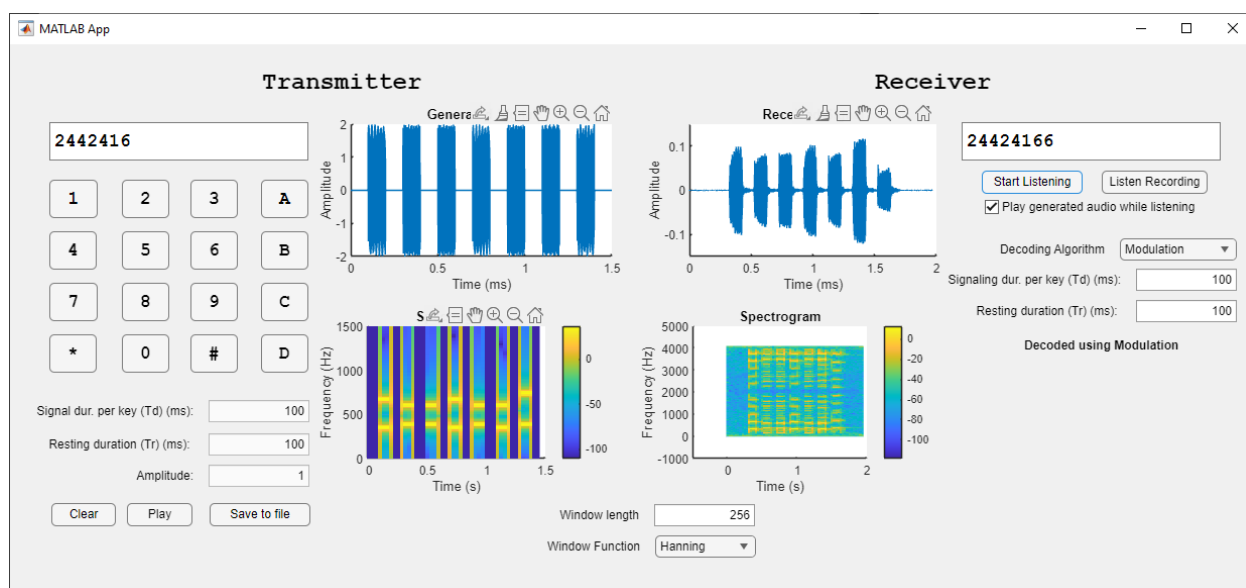
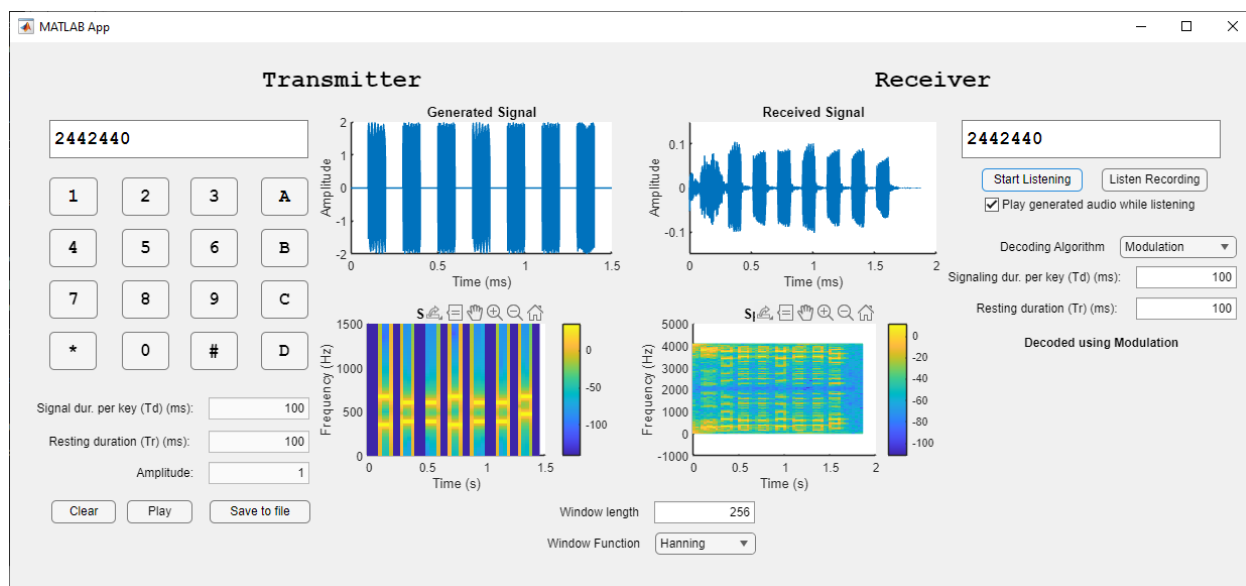
Because all algorithms did reasonably well in 250ms, 100ms and 40ms pulses in silent conditions, we did an additional test using 25ms pulses in a very noisy environment. In this test, the spectrogram-based approach became unusable as the spectrogram became unreadable and very noisy. Decoding using modulation was reasonably successful, and the filter banks did an excellent job, decoding the signal perfectly.

Tests Using Modulation

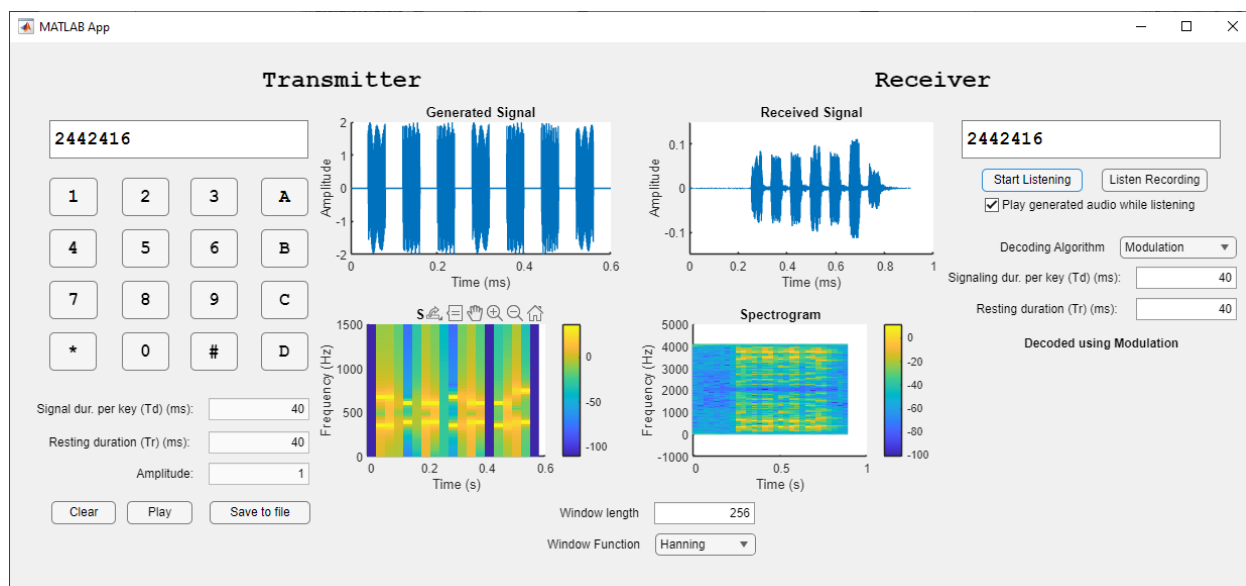
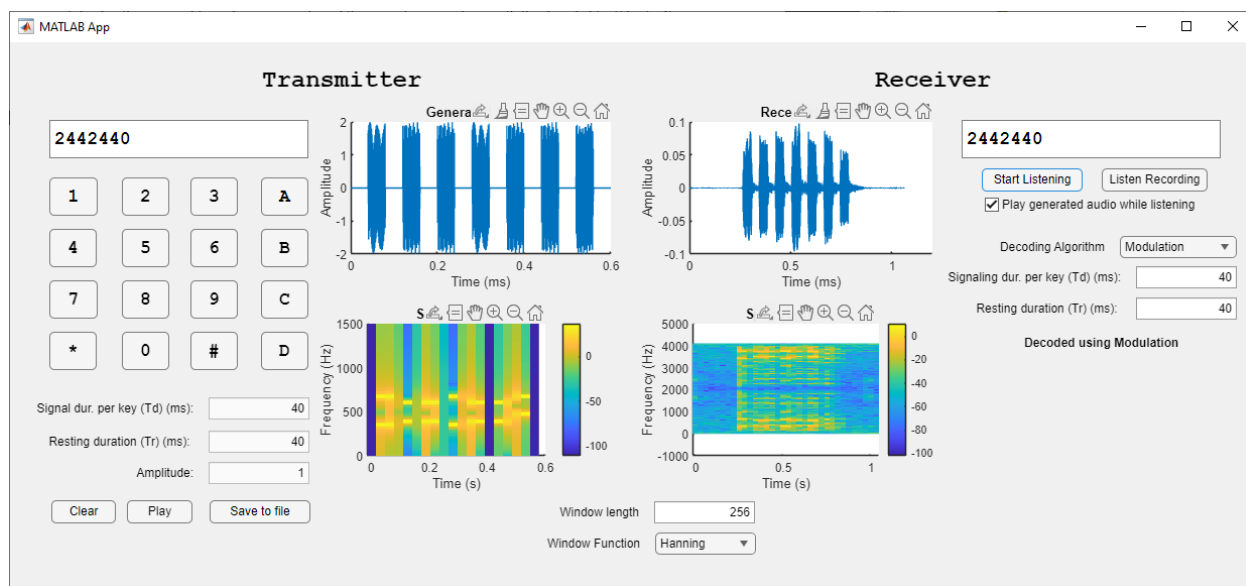
250ms with Modulation



100ms with Modulation

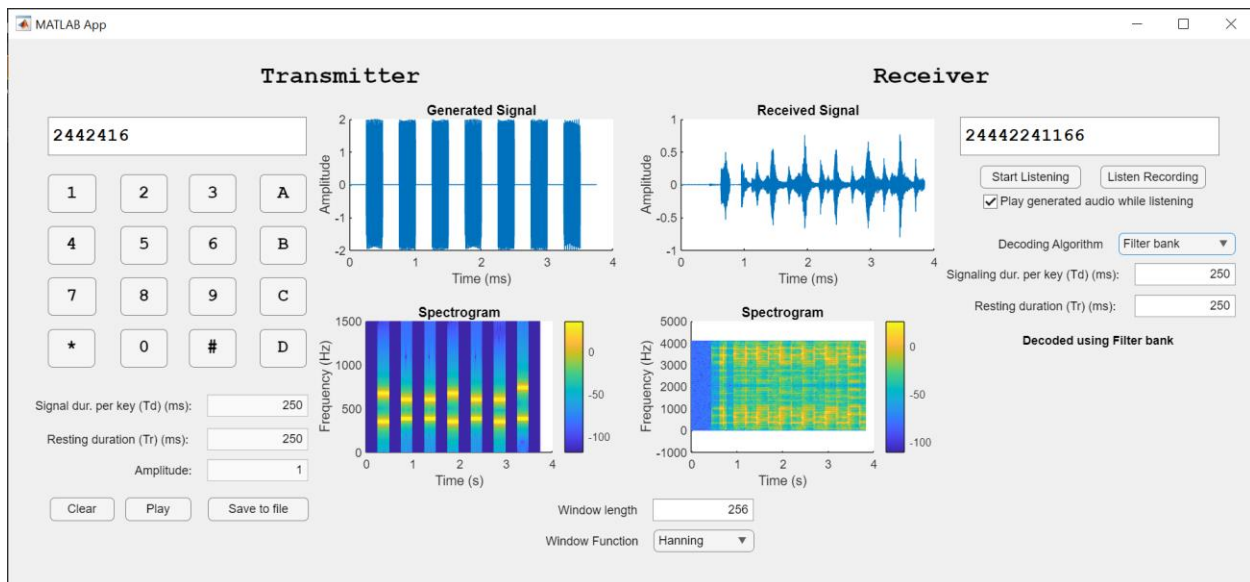
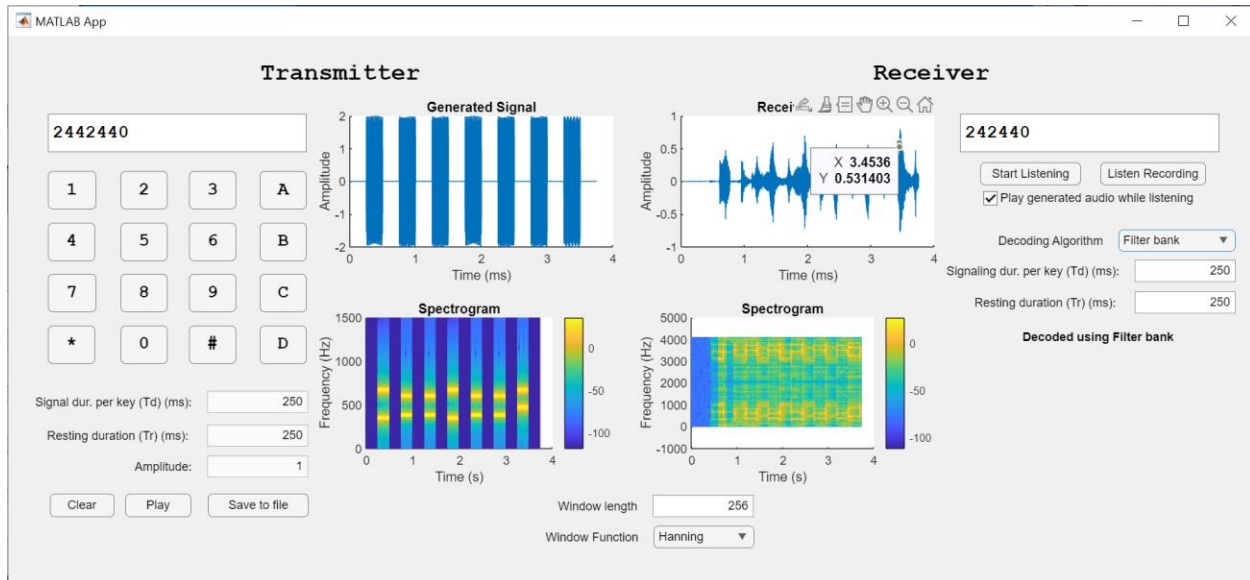


40ms with Modulation

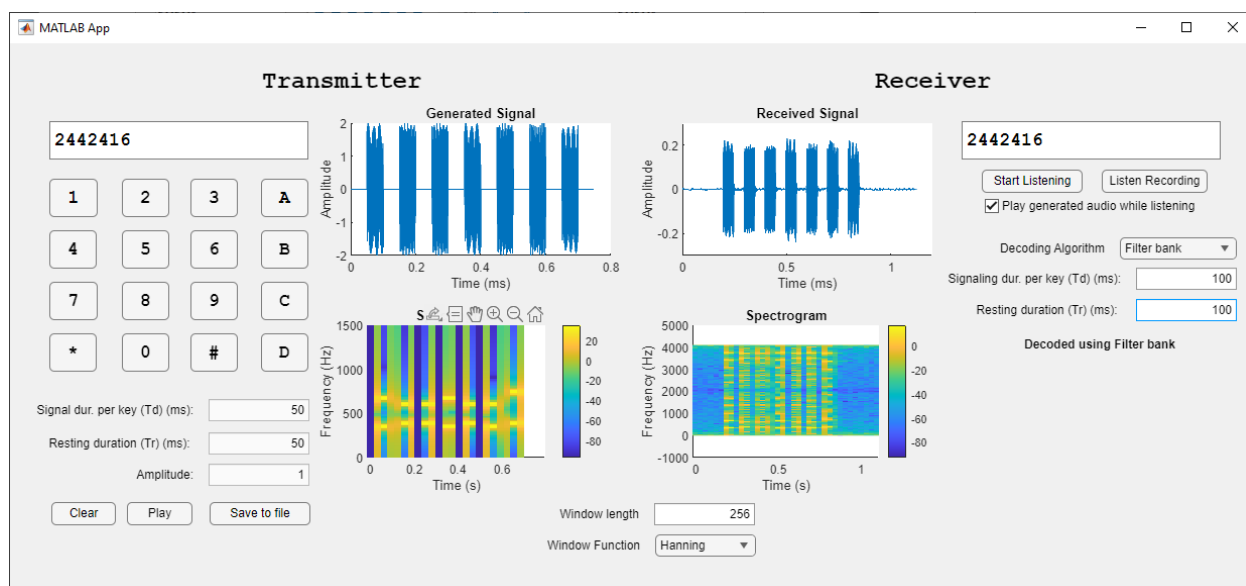
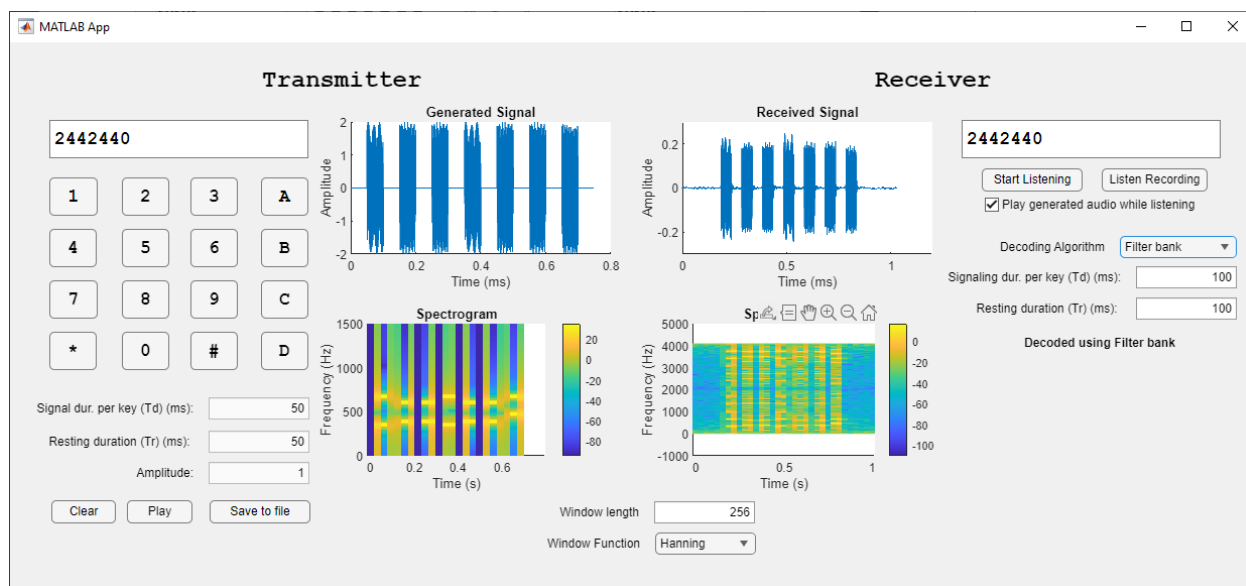


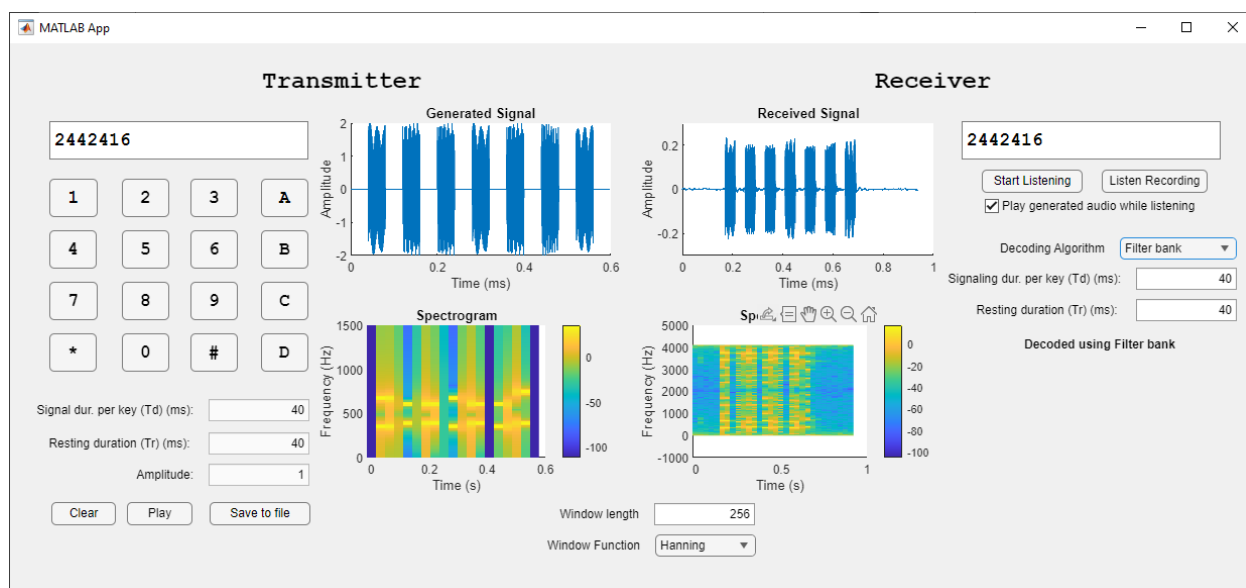
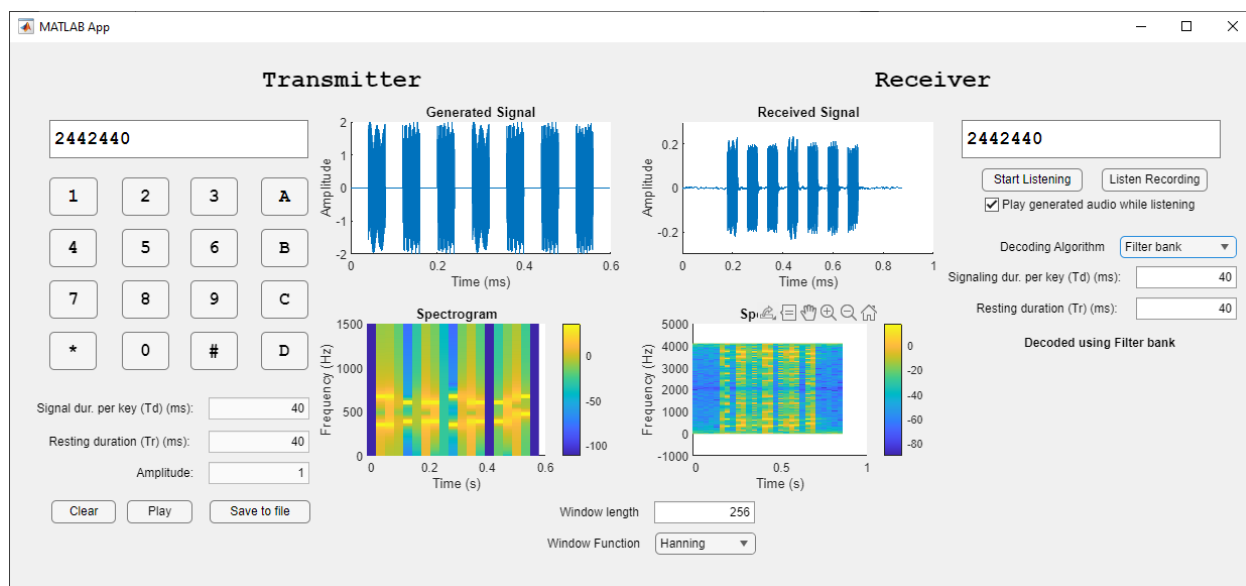
Tests Using Filter Banks

250ms with Filter Banks



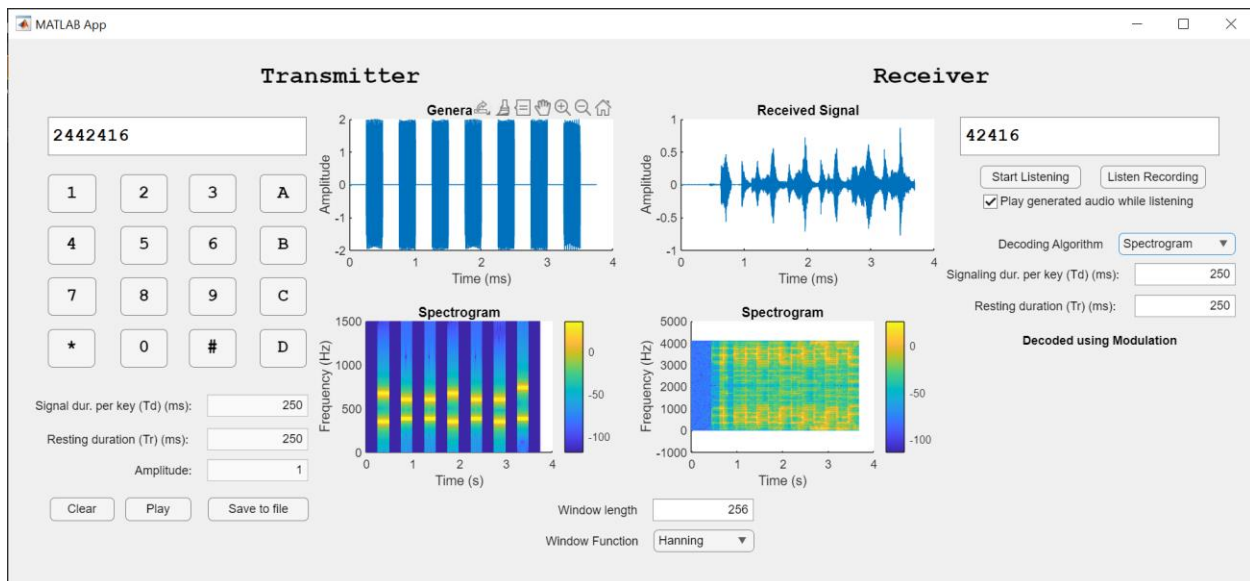
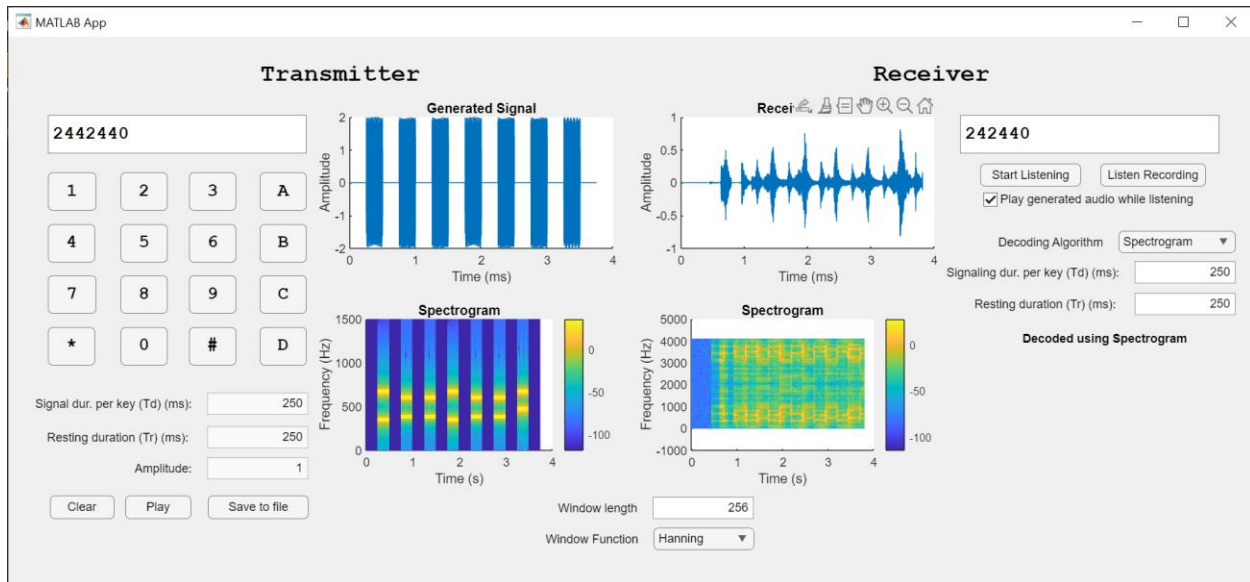
100ms with Filter Banks



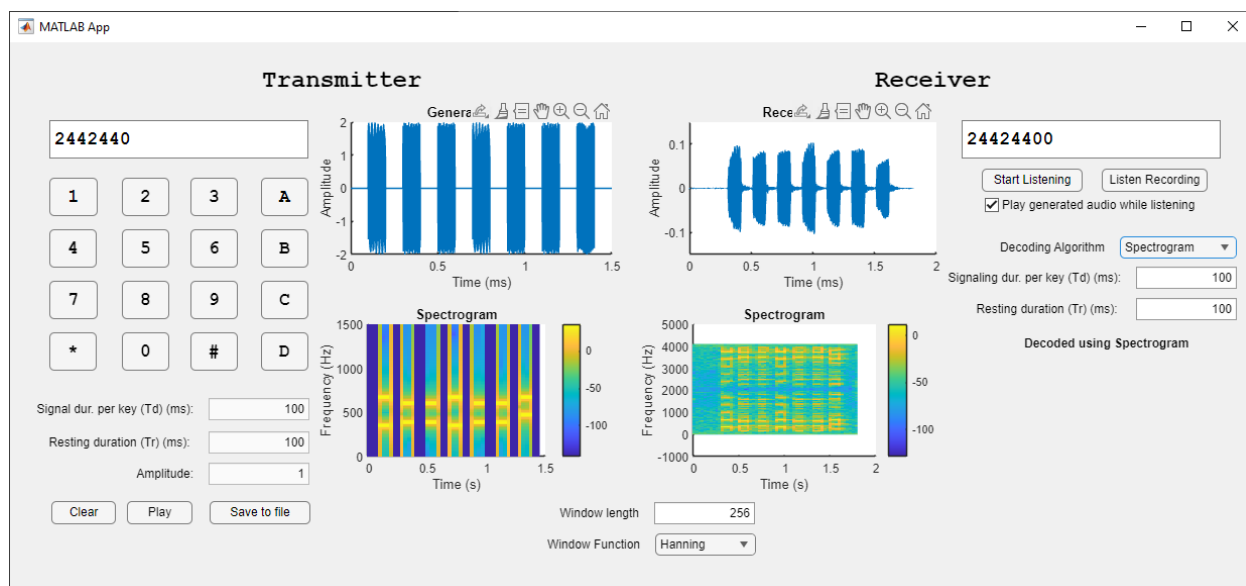
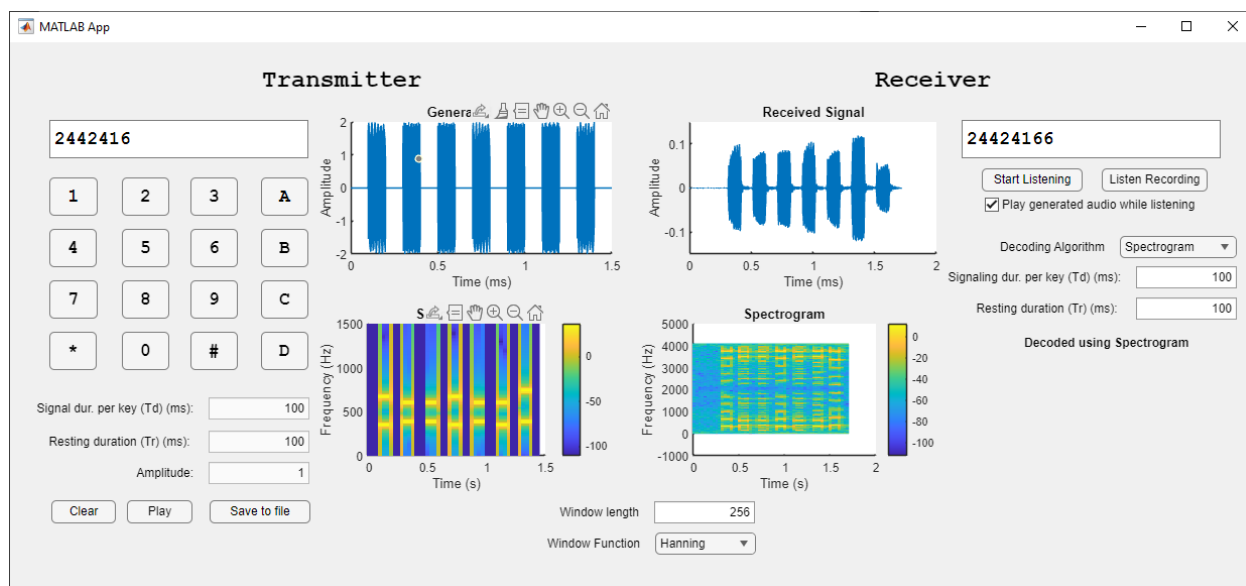
40ms with Filter Banks

Tests Using Spectrogram

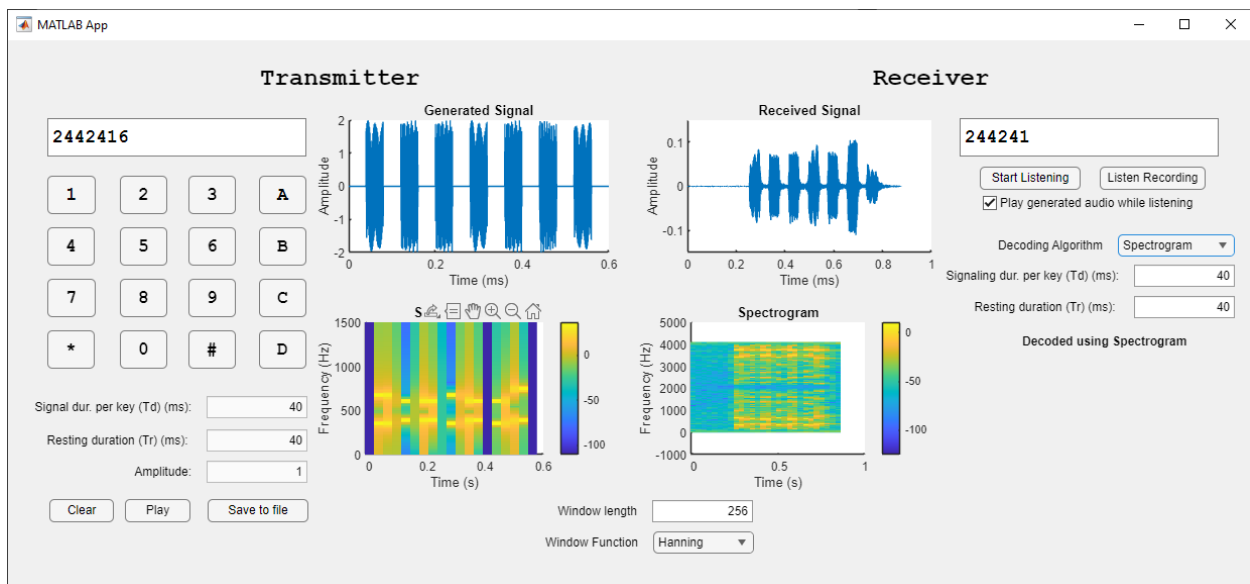
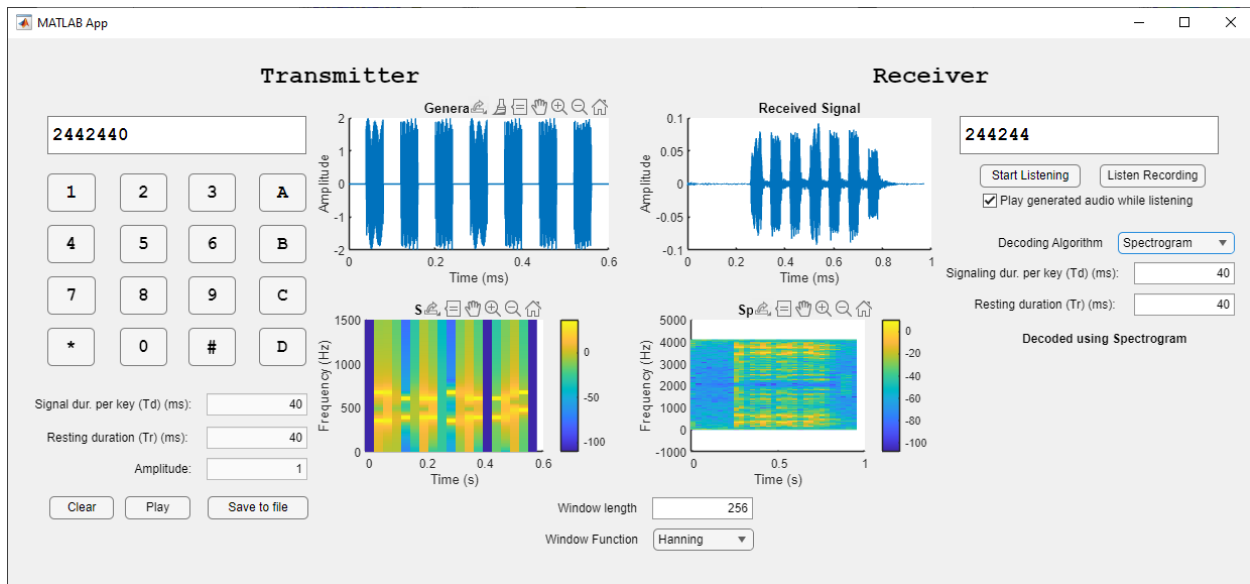
250ms with Spectrogram



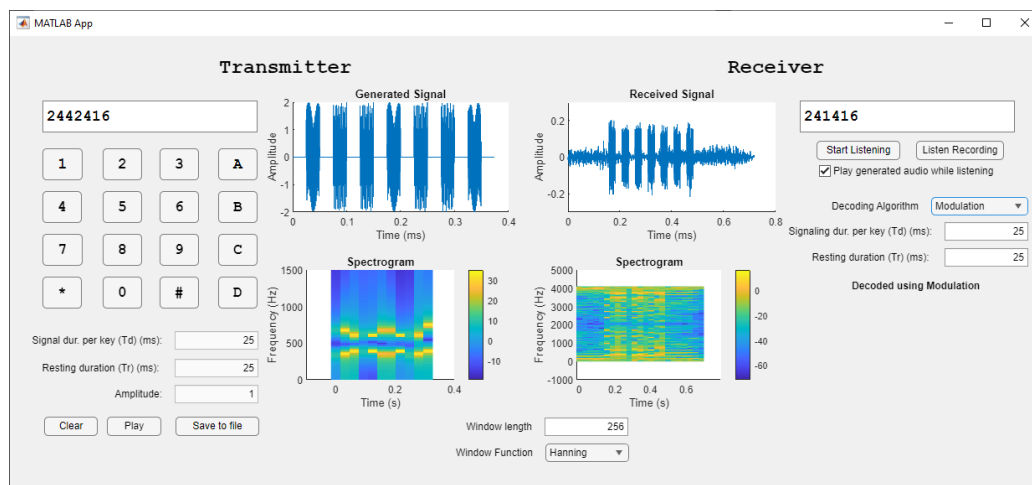
100ms with Spectrogram



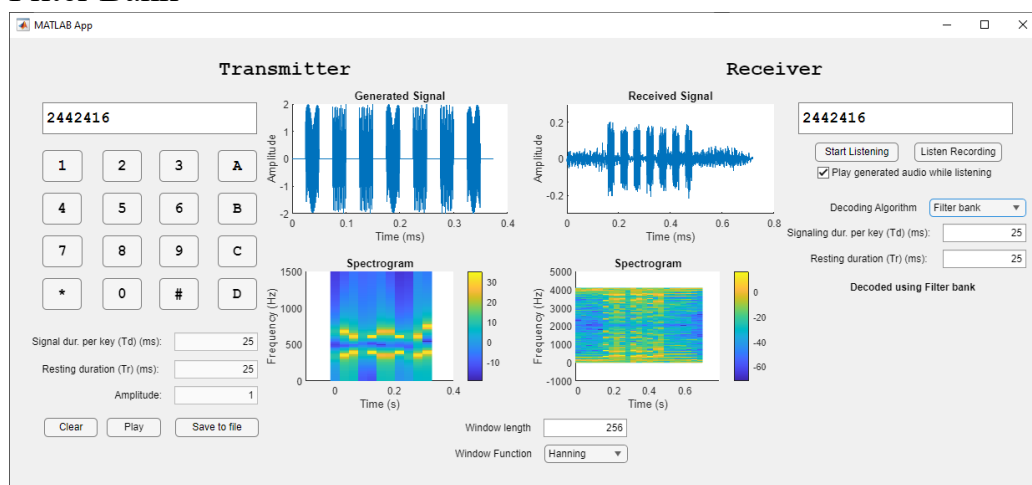
40ms with Spectrogram



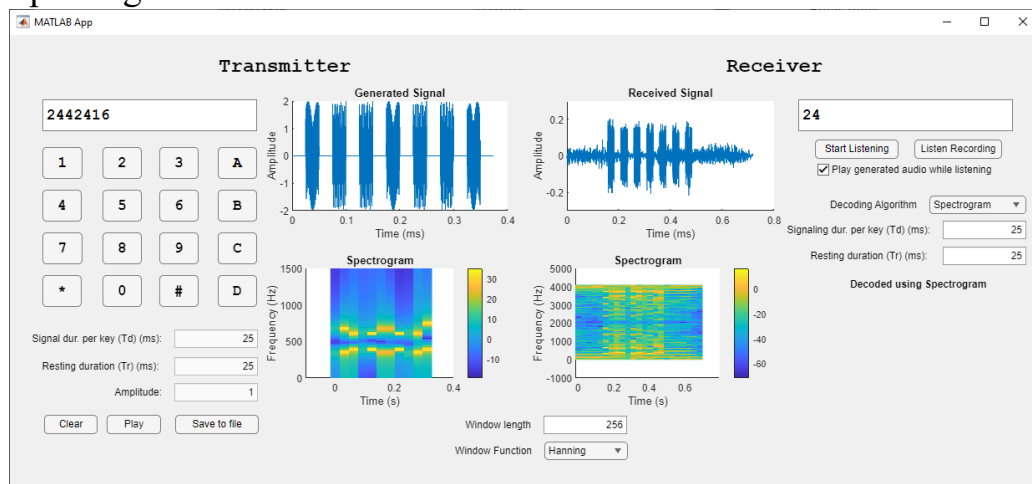
Bonus: $T_d = 25\text{ms}$ and $T_r = 25\text{ms}$ in a Noisy Environment Modulation



Filter Bank



Spectrogram



MATLAB Code

```

classdef termproject24424162442440 < matlab.apps.AppBase

    % Properties that correspond to app components
    properties (Access = public)
        UIFigure                                matlab.ui.Figure
        Noneedto recordanother timeLabel         matlab.ui.control.Label
        Itispossibletochangedecodingalgorithm after listeningLabel
matlab.ui.control.Label
        DecodeSuccess                           matlab.ui.control.Label
        PlayGenCheck                            matlab.ui.control.CheckBox
        ListenRecordingButton                    matlab.ui.control.Button
        DecodingAlgorithmDropDown                matlab.ui.control.DropDown
        DecodingAlgorithmDropDownLabel           matlab.ui.control.Label
        ReceivedKeys                             matlab.ui.control.EditField
        StartListeningButton                     matlab.ui.control.Button
        TrReceived                              matlab.ui.control.NumericEditField
        RestingdurationTrmsEditFieldLabel_2      matlab.ui.control.Label
        TdReceived                             matlab.ui.control.NumericEditField
        SignalingdurperkeyTdmsEditFieldLabel     matlab.ui.control.Label
        WindowFunctionDropDown                   matlab.ui.control.DropDown
        WindowFunctionDropDownLabel              matlab.ui.control.Label
        WindowlengthEditField                   matlab.ui.control.NumericEditField
        WindowlengthEditFieldLabel              matlab.ui.control.Label
        AmplitudeEditField                      matlab.ui.control.NumericEditField
        AmplitudeEditFieldLabel                 matlab.ui.control.Label
        Tr                                       matlab.ui.control.NumericEditField
        RestingdurationTrmsEditFieldLabel        matlab.ui.control.Label
        Td                                       matlab.ui.control.NumericEditField
        SignaldurperkeyTdmsEditFieldLabel        matlab.ui.control.Label
        ReceiverLabel                           matlab.ui.control.Label
        TransmitterLabel                        matlab.ui.control.Label
        PlayButton                              matlab.ui.control.Button
        SavetofileButton                         matlab.ui.control.Button
        ClearButton                             matlab.ui.control.Button
        PressedKeys                             matlab.ui.control.EditField
        DButton                                matlab.ui.control.Button
        CButton                                matlab.ui.control.Button
        BButton                                matlab.ui.control.Button
        AButton                                matlab.ui.control.Button
        Button_hash                             matlab.ui.control.Button
        Button_0                               matlab.ui.control.Button
        Button_star                             matlab.ui.control.Button
        Button_9                               matlab.ui.control.Button
        Button_8                               matlab.ui.control.Button
        Button_7                               matlab.ui.control.Button
        Button_6                               matlab.ui.control.Button
        Button_5                               matlab.ui.control.Button
        Button_4                               matlab.ui.control.Button
        Button_3                               matlab.ui.control.Button
        Button_2                               matlab.ui.control.Button
        Button_1                               matlab.ui.control.Button
    end

```

```

RecSpect                                matlab.ui.control.UIAxes
RecSig                                  matlab.ui.control.UIAxes
GenSpect                               matlab.ui.control.UIAxes
GenSig                                 matlab.ui.control.UIAxes
end

properties (Access = private)
    AudioData % Generated signals are added to this as the buttons are pressed
    SamplingRate = 8192 % Sampling rate of the generated signals (default 8000)
    Recorder % audiorecorder object for microphone input
    MicTimer % Timer for real-time processing
    listenedAudio
    trRec
    tdRec
end

methods (Access = private)

    function sine = makeSine(app, frequency)

        amplitude = app.AmplitudeEditField.Value;
        length = app.Td.Value / 1000;

        timeVector = linspace(0, length, app.SamplingRate*length);

        % Create sinusoidal signal
        sine = amplitude * sin(2 * pi * frequency * timeVector);
        %app.YouhaventprovidedaninputyetLabel.Text = ['Sine signal with freq '
num2str(frequency) ' Hz is created. Length: ' num2str(sampleCount) ' samples, '
num2str(timeInSec) ' seconds.'];
    end

    function dtmf = makeDTMF(app, key)

        switch(key)
            case '1'
                dtmf = makeSine(app, 697) + makeSine(app, 1209);
            case '2'
                dtmf = makeSine(app, 697) + makeSine(app, 1336);
            case '3'
                dtmf = makeSine(app, 697) + makeSine(app, 1477);
            case 'A'
                dtmf = makeSine(app, 697) + makeSine(app, 1633);
            case '4'
                dtmf = makeSine(app, 770) + makeSine(app, 1209);
            case '5'
                dtmf = makeSine(app, 770) + makeSine(app, 1336);
            case '6'
                dtmf = makeSine(app, 770) + makeSine(app, 1477);
            case 'B'
                dtmf = makeSine(app, 770) + makeSine(app, 1633);
            case '7'
                dtmf = makeSine(app, 852) + makeSine(app, 1209);

```

```

        case '8'
            dtmf = makeSine(app, 852) + makeSine(app, 1336);
        case '9'
            dtmf = makeSine(app, 852) + makeSine(app, 1477);
        case 'C'
            dtmf = makeSine(app, 852) + makeSine(app, 1633);
        case '*'
            dtmf = makeSine(app, 941) + makeSine(app, 1209);
        case '0'
            dtmf = makeSine(app, 941) + makeSine(app, 1336);
        case '#'
            dtmf = makeSine(app, 941) + makeSine(app, 1477);
        case 'D'
            dtmf = makeSine(app, 941) + makeSine(app, 1633);
    end

    dtmf = [dtmf, zeros(1, round(app.Tr.Value/1000*app.SamplingRate))]; %
add the delay after DTMF

end

function none = buttonPushed(app, key)

    app.Tr.Editable = 'off';
    app.Td.Editable = 'off';
    app.AmplitudeEditField.Editable = 'off';

    if isempty(app.AudioData)
        app.AudioData = [app.AudioData, zeros(1,
round(app.Tr.Value/1000*app.SamplingRate))];
    end

    app.AudioData = [app.AudioData, app.makeDTMF(key)]; % add new signal to
AudioData
    sound(app.makeDTMF(key), app.SamplingRate);

    plot(app.GenSig, linspace(0, length(app.AudioData)/app.SamplingRate,
length(app.AudioData)), app.AudioData);

    app.plotSpect();
    app.GenSpect.YLim = [0 1500];

    app.PressedKeys.Value = [app.PressedKeys.Value key];
end

function none = plotSpect(app)
    cla(app.GenSpect);
    x = app.AudioData;
    sigma = 20;

    window_size = app.WindowlengthEditField.Value;
    fs = app.SamplingRate;
    num_segments = floor((length(x)) / (window_size));
    spectrogram_matrix = zeros(window_size, num_segments);

```

```

switch app.WindowFunctionDropDown.Value
    case 'Gauss'
        n = -(window_size-1)/2:(window_size-1)/2;
        window = exp(-0.5 * ((n' / sigma).^2));
        window = window / sqrt(sum(window.^2));
    case 'Rectangular'
        window = 1;
    case 'Hanning'
        window = hann(window_size);
    case 'Hamming'
        window = hamming(window_size);
    otherwise
        window = 1;
end

for i = 1:num_segments
    start_index = (i - 1) * (window_size) + 1;
    end_index = start_index + window_size - 1;

    segment = x(start_index:end_index) .* window';

    segment_fft = fft(segment, window_size);
    spectrogram_matrix(:, i) = abs(segment_fft);
end

spectrogram_matrix_db = 20 * log10(spectrogram_matrix);
time_axis = linspace(0, (num_segments-1)*window_size/fs, num_segments);
frequency_axis = linspace(0, fs/2, window_size);

imagesc(app.GenSpect, time_axis, frequency_axis, spectrogram_matrix_db);
axis(app.GenSpect, 'xy');
title(app.GenSpect, 'Spectrogram');
xlabel(app.GenSpect, 'Time (s)');
ylabel(app.GenSpect, 'Frequency (Hz)');
colorbar(app.GenSpect);
end

function decodedKeys = modulationDecode(app)
    f=2.*pi.*([697,770,852,941,1209,1336,1477,1633]);
    k=['1','2','3','A','4','5','6','B','7','8','9','C','*','0','#','D'];
    fs=app.SamplingRate;

    st=5;
    y=app.listenedAudio;
    Fs=fs;

    o=zeros(length(y),length(f));

    sine_domain=0:1/fs:(length(y)/fs-1/fs);
    t=zeros(length(y),1);

    for i=1:length(f)
        modulation=0.5*cos((f(i)+100).*sine_domain);

```

```

        s=y'.*modulation;
        Hd=app.MakeModFilter();
        t=filter(Hd,s);
        t=t.*t;
        o(:,i)=movmean(t,200);
    end

    st=max(max(o))/st;
    g=[];
    t1=zeros(4,1);
    t2=zeros(4,1);
    s=0;

    for i=1:length(o)
        if sum(o(i,:))<st
            if s==0
                t1=zeros(4,1);
                t2=zeros(4,1);
            else
                [m,f1]=max(t1);
                [m,f2]=max(t2);
                g=[g,k(f1,f2)];
                s=0;
            end
        else
            [m,f1]=max(o(i,1:4));
            [m,f2]=max(o(i,5:end));
            t1(f1)=t1(f1)+1;
            t2(f2)=t2(f2)+1;
            s=1;

            if i==length(o)-1
                [m,f1]=max(t1);
                [m,f2]=max(t2);
                g=[g,k(f1,f2)];
                s=0;
            end
        end
    end

    decodedKeys=g;
    app.DecodeSuccess.Text="Decoded using Modulation";
    disp("Decoded using modulation")
    disp(g)
end

```

```

function filt = MakeModFilter(app)

    Fs = app.SamplingRate; % Sampling Frequency

    Fpass = 30; % Passband Frequency
    Fstop = 50; % Stopband Frequency
    Apass = 1; % Passband Ripple (dB)

```



```

    Astop = 80;           % Stopband Attenuation (dB)
    match = 'stopband'; % Band to match exactly

    h = fdesign.lowpass(Fpass, Fstop, Apass, Astop, Fs);
    filt = design(h, 'butter', 'MatchExactly', match);

end

function none = startListening(app)

    app.Recorder = audiorecorder(app.SamplingRate, 16, 1); % Adjust as needed
    record(app.Recorder);

end

function none = stopListening(app)

    stop(app.Recorder)
    app.listenedAudio = getaudiodata(app.Recorder);

end

function none = plotListenedSpect(app)
    cla(app.RecSpect);
    x = app.listenedAudio';
    sigma = 20;

    % Parameters for the spectrogram
    window_size = app.WindowlengthEditField.Value; % 2M
    fs = app.SamplingRate;

    % Calculate the number of segments
    num_segments = floor((length(x)) / (window_size));

    % Initialize the spectrogram matrix
    spectrogram_matrix = zeros(window_size, num_segments);

    switch app.WindowFunctionDropDown.Value
        case 'Gauss'
            n = -(window_size-1)/2:(window_size-1)/2;
            window = exp(-0.5 * ((n' / sigma).^2));

            % Normalize the window to have unit energy
            window = window / sqrt(sum(window.^2));
        case 'Rectangular'
            window = 1;
        case 'Hanning'
            window = hann(window_size);
        case 'Hamming'
            window = hamming(window_size);
        otherwise
            window = 1;
    end
end

```

```

for i = 1:num_segments
    start_index = (i - 1) * (window_size) + 1;
    end_index = start_index + window_size - 1;

    segment = x(start_index:end_index) .* window';

    segment_fft = fft(segment, window_size);

    spectrogram_matrix(:, i) = abs(segment_fft);
end

spectrogram_matrix_db = 20 * log10(spectrogram_matrix);

time_axis = linspace(0, (num_segments-1)*window_size/fs, num_segments);
frequency_axis = linspace(0, fs/2, window_size);

imagesc(app.RecSpect, time_axis, frequency_axis, spectrogram_matrix_db);
axis (app.RecSpect, 'xy');
title(app.RecSpect, 'Spectrogram');
xlabel(app.RecSpect, 'Time (s)');
ylabel(app.RecSpect, 'Frequency (Hz)');
colorbar(app.RecSpect);
end

function decodedKeys = filterbankDecode(app)
    freqs = [697,770,852,941,1209,1336,1477,1633];
    keys = ['1','2','3','A','4','5','6','B','7','8','9','C','*','0','#','D'];
    fs = app.SamplingRate;

    silence_diff_threshold = 5;

    y = app.listenedAudio;

    filtered_output = zeros(length(y),length(freqs));
    temp = zeros(length(y),1);
    Rp = 3;
    Rs = 40;

    for i = 1:length(freqs)
        Hd = myfilter(app, freqs(i)-25, freqs(i)+25);
        temp = filter(Hd,y);
        temp = temp.*temp;
        filtered_output(:,i) = movmean(temp,200);
    end

    silence_threshold = max(max(filtered_output))/silence_diff_threshold;
    guesses = [];
    total1 = zeros(4,1);
    total2 = zeros(4,1);
    state = 0;

    for i = 1:length(filtered_output)
        if sum(filtered_output(i,:)) < silence_threshold
            if state == 0
                total1 = zeros(4,1);
            end
        end
    end

```

```

        total2 = zeros(4,1);
    else
        [m,f1] = max(total1);
        [m,f2] = max(total2);
        guesses = [guesses,keys(f1,f2)];
        state = 0;
    end
else
    [m,f1] = max(filtered_output(i,1:4));
    [m,f2] = max(filtered_output(i,5:end));
    total1(f1)=total1(f1)+1;
    total2(f2)=total2(f2)+1;
    state = 1;

    if i == length(filtered_output)-1
        [m,f1] = max(total1);
        [m,f2] = max(total2);
        guesses = [guesses,keys(f1,f2)];
        state = 0;
    end
end
end

decodedKeys = guesses;
app.DecodeSuccess.Text = "Decoded using Filter bank";
end

% this function generates the butterworth bandpass filters we use
% for filter bank decoding
function Hd = myfilter(app, Fc1, Fc2)
    Fs = app.SamplingRate;
    N = 10; %% this is the order of the filter, we can change it if needed
    h = fdesign.bandpass('N,F3dB1,F3dB2',N,Fc1,Fc2,Fs);
    Hd = design(h, 'butter');
end

function decodedKeys = spectroDecode(app)

    x = app.listenedAudio
    fs = app.SamplingRate

    pulse_length = app.TrReceived.Value;
    pulse_delay = app.TdReceived.Value;

    window_size = 256;
    overlap = 64;
    [spectrogram_data, frequencies, times] = spectrogram(x, window_size,
overlap, window_size, fs);

    low_freq_group = [697, 770, 852, 941];
    high_freq_group = [1209, 1336, 1477, 1633];

```

```

dtmf_digits = ['1', '2', '3', 'A';
               '4', '5', '6', 'B';
               '7', '8', '9', 'C';
               '*', '0', '#', 'D'];

decoded_sequence = '';

for t = 1:length(times)
    start_index = round(t * fs - pulse_delay * fs);
    end_index = start_index + round(pulse_length * fs) - 1;

    if end_index > length(x)
        break;
    end

    current_pulse = x(start_index:end_index);

    [pulse_spectrogram, pulse_frequencies, ~] =
spectrogram(current_pulse, window_size, overlap, window_size, fs);

    % find the indices of the two highest frequencies in the pulse
spectrogram
    [~, sorted_indices] = sort(pulse_spectrogram, 'descend');
    highest_indices = sorted_indices(1:2);

    highest_freqs = [pulse_frequencies(highest_indices(1));
pulse_frequencies(highest_indices(2))];

    [~, low_index] = min(abs(low_freq_group - highest_freqs(1)));
    [~, high_index] = min(abs(high_freq_group - highest_freqs(2)));

    decoded_digit = dtmf_digits(low_index, high_index);
    decoded_sequence = strcat(decoded_sequence, decoded_digit);
end

decodedKeys = decoded_sequence;

end
end

% Callbacks that handle component events
methods (Access = private)

    % Code that executes after component creation
    function startupFcn(app)

end

% Button pushed function: Button_1
function Button_1Pushed(app, event)

```

```
        app.buttonPushed('1');
    end

    % Button pushed function: Button_2
    function Button_2Pushed(app, event)
        app.buttonPushed('2');
    end

    % Button pushed function: Button_3
    function Button_3Pushed(app, event)
        app.buttonPushed('3');
    end

    % Button pushed function: Button_4
    function Button_4Pushed(app, event)
        app.buttonPushed('4');
    end

    % Button pushed function: Button_5
    function Button_5Pushed(app, event)
        app.buttonPushed('5');
    end

    % Button pushed function: Button_6
    function Button_6Pushed(app, event)
        app.buttonPushed('6');
    end

    % Button pushed function: Button_7
    function Button_7Pushed(app, event)
        app.buttonPushed('7');
    end

    % Button pushed function: Button_8
    function Button_8Pushed(app, event)
        app.buttonPushed('8');
    end

    % Button pushed function: Button_9
    function Button_9Pushed(app, event)
        app.buttonPushed('9');
    end

    % Button pushed function: Button_star
    function Button_starPushed(app, event)
        app.buttonPushed('*');
    end

    % Button pushed function: Button_0
    function Button_0Pushed(app, event)
        app.buttonPushed('0');
    end

    % Button pushed function: Button_hash
    function Button_hashPushed(app, event)
```

```

        app.buttonPushed('#');
    end

    % Button pushed function: AButton
    function AButtonPushed(app, event)
        app.buttonPushed('A');
    end

    % Button pushed function: BButton
    function BButtonPushed(app, event)
        app.buttonPushed('B');
    end

    % Button pushed function: CButton
    function CButtonPushed(app, event)
        app.buttonPushed('C');
    end

    % Button pushed function: DButton
    function DButtonPushed(app, event)
        app.buttonPushed('D');
    end

    % Button pushed function: ClearButton
    function ClearButtonPushed(app, event)
        app.AudioData = [];
        cla(app.GenSig);
        cla(app.GenSpect);
        app.PressedKeys.Value = '';

        app.Tr.Editable = 'on';
        app.Td.Editable = 'on';
        app.AmplitudeEditField.Editable = 'on';
    end

    % Button pushed function: PlayButton
    function PlayButtonPushed(app, event)
        if isempty(app.AudioData)
        else
            sound(app.AudioData, app.SamplingRate)
        end
    end

    % Value changed function: WindowFunctionDropDown
    function WindowFunctionDropDownValueChanged(app, event)
        value = app.WindowFunctionDropDown.Value;
        app.plotSpect();
    end

    % Value changed function: WindowlengthEditField
    function WindowlengthEditFieldValueChanged(app, event)
        value = app.WindowlengthEditField.Value;
        app.plotSpect();
    end
end

```

```

% Button pushed function: SavetofileButton
function SavetofileButtonPushed(app, event)
    % Check if there is any audio data to save
    if isempty(app.AudioData)
        disp('No audio data to save. ');
        return;
    end

    % Prompt user for a file name
    [filename, filepath] = uiputfile('*.wav', 'Save Audio Data As');

    % Check if the user canceled the file selection
    if isequal(filename, 0) || isequal(filepath, 0)
        disp('File save operation canceled. ');
        return;
    end

    % Combine the filename and filepath
    fullFilePath = fullfile(filepath, filename);

    % Save the audio data to a WAV file
    try
        audiowrite(fullFilePath, app.AudioData, app.SamplingRate);
        disp(['Audio data saved to ' fullFilePath]);
    catch
        disp('Error saving audio data. ');
    end
end

% Button pushed function: StartListeningButton
function StartListeningButtonPushed(app, event)

    if strcmp(app.StartListeningButton.Text, 'Start Listening')
        if(app.PlayGenCheck.Value)
            sound(app.AudioData, app.SamplingRate)
        end

        app.startListening();
        app.StartListeningButton.Text = 'Stop Listening';
    elseif strcmp(app.StartListeningButton.Text, 'Stop Listening')
        app.StartListeningButton.Text = 'Start Listening';
        app.stopListening();
        plot(app.RecSig, linspace(0,
length(app.listenedAudio)/app.SamplingRate, length(app.listenedAudio)),
app.listenedAudio);
        app.plotListenedSpect();
        app.DecodingAlgorithmDropDown.Value = "Modulation";
        app.ReceivedKeys.Value=app.modulationDecode();
        app.DecodeSuccess.Visible="on";
    end

end

end

% Value changed function: TrReceived

```

```

function TrReceivedValueChanged(app, event)
    app.trRec = app.TrReceived.Value;
end

% Value changed function: TdReceived
function TdReceivedValueChanged(app, event)
    app.tdRec = app.TdReceived.Value;
end

% Value changed function: DecodingAlgorithmDropDown
function DecodingAlgorithmDropDownValueChanged(app, event)
    if app.DecodingAlgorithmDropDown.Value == "Modulation"
        app.ReceivedKeys.Value = app.modulationDecode();
    elseif app.DecodingAlgorithmDropDown.Value == "Filter bank"
        app.ReceivedKeys.Value = app.filterbankDecode();
    elseif app.DecodingAlgorithmDropDown.Value == "Spectrogram"
        app.ReceivedKeys.Value = app.modulationDecode();
        app.DecodeSuccess.Text = "Decoded using Spectrogram";
    end

end

% Button pushed function: ListenRecordingButton
function ListenRecordingButtonPushed(app, event)
    sound(app.listenedAudio, app.SamplingRate);
end
end

% Component initialization
methods (Access = private)

% Create UIFigure and components
function createComponents(app)

    % Create UIFigure and hide until all components are created
    app.UIFigure = uifigure('Visible', 'off');
    app.UIFigure.Position = [100 100 1236 544];
    app.UIFigure.Name = 'MATLAB App';

    % Create GenSig
    app.GenSig = uiaxes(app.UIFigure);
    title(app.GenSig, 'Generated Signal')
    xlabel(app.GenSig, 'Time (ms)')
    ylabel(app.GenSig, 'Amplitude')
    zlabel(app.GenSig, 'Z')
    app.GenSig.Position = [306 300 300 185];

    % Create GenSpect
    app.GenSpect = uiaxes(app.UIFigure);
    title(app.GenSpect, 'Spectrogram of generated signal')
    xlabel(app.GenSpect, 'Time (ms)')
    ylabel(app.GenSpect, 'Frequency (Hz)')
    app.GenSpect.Position = [306 100 300 185];

```



```

% Create RecSig
app.RecSig = uiaxes(app.UIFigure);
title(app.RecSig, 'Received Signal')
xlabel(app.RecSig, 'Time (ms)')
ylabel(app.RecSig, 'Amplitude')
zlabel(app.RecSig, 'Z')
app.RecSig.Position = [625 300 300 185];

% Create RecSpect
app.RecSpect = uiaxes(app.UIFigure);
title(app.RecSpect, 'Spectrogram of received signal')
xlabel(app.RecSpect, 'Time (ms)')
ylabel(app.RecSpect, 'Frequency (Hz)')
zlabel(app.RecSpect, 'Z')
app.RecSpect.Position = [625 100 300 185];

% Create Button_1
app.Button_1 = uibutton(app.UIFigure, 'push');
app.Button_1.ButtonPushedFcn = createCallbackFcn(app, @Button_1Pushed,
true);

app.Button_1.FontName = 'Courier New';
app.Button_1.FontSize = 18;
app.Button_1.FontWeight = 'bold';
app.Button_1.Position = [40 373 48 38];
app.Button_1.Text = '1';

% Create Button_2
app.Button_2 = uibutton(app.UIFigure, 'push');
app.Button_2.ButtonPushedFcn = createCallbackFcn(app, @Button_2Pushed,
true);

app.Button_2.FontName = 'Courier New';
app.Button_2.FontSize = 18;
app.Button_2.FontWeight = 'bold';
app.Button_2.Position = [112 373 47 38];
app.Button_2.Text = '2';

% Create Button_3
app.Button_3 = uibutton(app.UIFigure, 'push');
app.Button_3.ButtonPushedFcn = createCallbackFcn(app, @Button_3Pushed,
true);

app.Button_3.FontName = 'Courier New';
app.Button_3.FontSize = 18;
app.Button_3.FontWeight = 'bold';
app.Button_3.Position = [180 373 47 38];
app.Button_3.Text = '3';

% Create Button_4
app.Button_4 = uibutton(app.UIFigure, 'push');
app.Button_4.ButtonPushedFcn = createCallbackFcn(app, @Button_4Pushed,
true);

app.Button_4.FontName = 'Courier New';
app.Button_4.FontSize = 18;
app.Button_4.FontWeight = 'bold';
app.Button_4.Position = [40 322 47 38];
app.Button_4.Text = '4';

```

```

    % Create Button_5
    app.Button_5 = uibutton(app.UIFigure, 'push');
    app.Button_5.ButtonPushedFcn = createCallbackFcn(app, @Button_5Pushed,
true);

    app.Button_5.FontName = 'Courier New';
    app.Button_5.FontSize = 18;
    app.Button_5.FontWeight = 'bold';
    app.Button_5.Position = [112 322 47 38];
    app.Button_5.Text = '5';

    % Create Button_6
    app.Button_6 = uibutton(app.UIFigure, 'push');
    app.Button_6.ButtonPushedFcn = createCallbackFcn(app, @Button_6Pushed,
true);

    app.Button_6.FontName = 'Courier New';
    app.Button_6.FontSize = 18;
    app.Button_6.FontWeight = 'bold';
    app.Button_6.Position = [180 322 47 38];
    app.Button_6.Text = '6';

    % Create Button_7
    app.Button_7 = uibutton(app.UIFigure, 'push');
    app.Button_7.ButtonPushedFcn = createCallbackFcn(app, @Button_7Pushed,
true);

    app.Button_7.FontName = 'Courier New';
    app.Button_7.FontSize = 18;
    app.Button_7.FontWeight = 'bold';
    app.Button_7.Position = [40 271 48 38];
    app.Button_7.Text = '7';

    % Create Button_8
    app.Button_8 = uibutton(app.UIFigure, 'push');
    app.Button_8.ButtonPushedFcn = createCallbackFcn(app, @Button_8Pushed,
true);

    app.Button_8.FontName = 'Courier New';
    app.Button_8.FontSize = 18;
    app.Button_8.FontWeight = 'bold';
    app.Button_8.Position = [112 271 47 38];
    app.Button_8.Text = '8';

    % Create Button_9
    app.Button_9 = uibutton(app.UIFigure, 'push');
    app.Button_9.ButtonPushedFcn = createCallbackFcn(app, @Button_9Pushed,
true);

    app.Button_9.FontName = 'Courier New';
    app.Button_9.FontSize = 18;
    app.Button_9.FontWeight = 'bold';
    app.Button_9.Position = [180 271 47 38];
    app.Button_9.Text = '9';

    % Create Button_star
    app.Button_star = uibutton(app.UIFigure, 'push');
    app.Button_star.ButtonPushedFcn = createCallbackFcn(app,
@Button_starPushed, true);

```

```

app.Button_star.FontName = 'Courier New';
app.Button_star.FontSize = 18;
app.Button_star.FontWeight = 'bold';
app.Button_star.Position = [40 220 47 38];
app.Button_star.Text = '*';

% Create Button_0
app.Button_0 = uibutton(app.UIFigure, 'push');
app.Button_0.ButtonPushedFcn = createCallbackFcn(app, @Button_0Pushed,
true);

app.Button_0.FontName = 'Courier New';
app.Button_0.FontSize = 18;
app.Button_0.FontWeight = 'bold';
app.Button_0.Position = [112 220 47 38];
app.Button_0.Text = '0';

% Create Button_hash
app.Button_hash = uibutton(app.UIFigure, 'push');
app.Button_hash.ButtonPushedFcn = createCallbackFcn(app,
@Button_hashPushed, true);
app.Button_hash.FontName = 'Courier New';
app.Button_hash.FontSize = 18;
app.Button_hash.FontWeight = 'bold';
app.Button_hash.Position = [180 220 47 38];
app.Button_hash.Text = '#';

% Create AButton
app.AButton = uibutton(app.UIFigure, 'push');
app.AButton.ButtonPushedFcn = createCallbackFcn(app, @AButtonPushed,
true);

app.AButton.FontName = 'Courier New';
app.AButton.FontSize = 18;
app.AButton.FontWeight = 'bold';
app.AButton.Position = [250 373 47 38];
app.AButton.Text = 'A';

% Create BButton
app.BButton = uibutton(app.UIFigure, 'push');
app.BButton.ButtonPushedFcn = createCallbackFcn(app, @BButtonPushed,
true);

app.BButton.FontName = 'Courier New';
app.BButton.FontSize = 18;
app.BButton.FontWeight = 'bold';
app.BButton.Position = [250 322 47 38];
app.BButton.Text = 'B';

% Create CButton
app.CButton = uibutton(app.UIFigure, 'push');
app.CButton.ButtonPushedFcn = createCallbackFcn(app, @CButtonPushed,
true);

app.CButton.FontName = 'Courier New';
app.CButton.FontSize = 18;
app.CButton.FontWeight = 'bold';
app.CButton.Position = [250 271 47 38];
app.CButton.Text = 'C';

```

```

    % Create DButton
    app.DButton = uibutton(app.UIFigure, 'push');
    app.DButton.ButtonPushedFcn = createCallbackFcn(app, @DButtonPushed,
true);

    app.DButton.FontName = 'Courier New';
    app.DButton.FontSize = 18;
    app.DButton.FontWeight = 'bold';
    app.DButton.Position = [250 220 47 38];
    app.DButton.Text = 'D';

    % Create PressedKeys
    app.PressedKeys = uieditfield(app.UIFigure, 'text');
    app.PressedKeys.FontName = 'Courier New';
    app.PressedKeys.FontSize = 18;
    app.PressedKeys.FontWeight = 'bold';
    app.PressedKeys.Position = [40 430 257 38];

    % Create ClearButton
    app.ClearButton = uibutton(app.UIFigure, 'push');
    app.ClearButton.ButtonPushedFcn = createCallbackFcn(app,
@ClearButtonPushed, true);
    app.ClearButton.Position = [42 68 64 23];
    app.ClearButton.Text = 'Clear';

    % Create SavetofileButton
    app.SavetofileButton = uibutton(app.UIFigure, 'push');
    app.SavetofileButton.ButtonPushedFcn = createCallbackFcn(app,
@SavetofileButtonPushed, true);
    app.SavetofileButton.Position = [199 68 100 23];
    app.SavetofileButton.Text = 'Save to file';

    % Create PlayButton
    app.PlayButton = uibutton(app.UIFigure, 'push');
    app.PlayButton.ButtonPushedFcn = createCallbackFcn(app,
@PlayButtonPushed, true);
    app.PlayButton.Position = [117 68 65 23];
    app.PlayButton.Text = 'Play';

    % Create TransmitterLabel
    app.TransmitterLabel = uilabel(app.UIFigure);
    app.TransmitterLabel.FontName = 'Courier New';
    app.TransmitterLabel.FontSize = 24;
    app.TransmitterLabel.FontWeight = 'bold';
    app.TransmitterLabel.Position = [251 493 163 30];
    app.TransmitterLabel.Text = 'Transmitter';

    % Create ReceiverLabel
    app.ReceiverLabel = uilabel(app.UIFigure);
    app.ReceiverLabel.FontName = 'Courier New';
    app.ReceiverLabel.FontSize = 24;
    app.ReceiverLabel.FontWeight = 'bold';
    app.ReceiverLabel.Position = [859 493 120 30];
    app.ReceiverLabel.Text = 'Receiver';

```

```

% Create SignaldurperkeyTdmsEditFieldLabel
app.SignaldurperkeyTdmsEditFieldLabel = uilabel(app.UIFigure);
app.SignaldurperkeyTdmsEditFieldLabel.HorizontalAlignment = 'right';
app.SignaldurperkeyTdmsEditFieldLabel.Position = [23 171 160 22];
app.SignaldurperkeyTdmsEditFieldLabel.Text = 'Signal dur. per key (Td)
(ms)':'';

% Create Td
app.Td = uieditfield(app.UIFigure, 'numeric');
app.Td.Position = [198 171 100 22];
app.Td.Value = 50;

% Create RestingdurationTrmsEditFieldLabel
app.RestingdurationTrmsEditFieldLabel = uilabel(app.UIFigure);
app.RestingdurationTrmsEditFieldLabel.HorizontalAlignment = 'right';
app.RestingdurationTrmsEditFieldLabel.Position = [34 138 149 22];
app.RestingdurationTrmsEditFieldLabel.Text = 'Resting duration (Tr) (ms):
';

% Create Tr
app.Tr = uieditfield(app.UIFigure, 'numeric');
app.Tr.Position = [198 138 100 22];
app.Tr.Value = 50;

% Create AmplitudeEditFieldLabel
app.AmplitudeEditFieldLabel = uilabel(app.UIFigure);
app.AmplitudeEditFieldLabel.HorizontalAlignment = 'right';
app.AmplitudeEditFieldLabel.Position = [121 107 62 22];
app.AmplitudeEditFieldLabel.Text = 'Amplitude: ';

% Create AmplitudeEditField
app.AmplitudeEditField = uieditfield(app.UIFigure, 'numeric');
app.AmplitudeEditField.Position = [198 107 100 22];
app.AmplitudeEditField.Value = 1;

% Create WindowlengthEditFieldLabel
app.WindowlengthEditFieldLabel = uilabel(app.UIFigure);
app.WindowlengthEditFieldLabel.HorizontalAlignment = 'right';
app.WindowlengthEditFieldLabel.Position = [541 68 84 22];
app.WindowlengthEditFieldLabel.Text = 'Window length';

% Create WindowlengthEditField
app.WindowlengthEditField = uieditfield(app.UIFigure, 'numeric');
app.WindowlengthEditField.ValueChangedFcn = createCallbackFcn(app,
@WindowlengthEditFieldValueChanged, true);
app.WindowlengthEditField.Position = [640 68 100 22];
app.WindowlengthEditField.Value = 256;

% Create WindowFunctionDropDownLabel
app.WindowFunctionDropDownLabel = uilabel(app.UIFigure);
app.WindowFunctionDropDownLabel.HorizontalAlignment = 'right';
app.WindowFunctionDropDownLabel.Position = [529 37 97 22];
app.WindowFunctionDropDownLabel.Text = 'Window Function';

% Create WindowFunctionDropDown

```

```

    app.WindowFunctionDropDown = uidropdown(app.UIFigure);
    app.WindowFunctionDropDown.Items = {'Hanning', 'Hamming', 'Gauss',
'Rectangular'};
    app.WindowFunctionDropDown.ValueChangedFcn = createCallbackFcn(app,
@WindowFunctionDropDownValueChanged, true);
    app.WindowFunctionDropDown.Position = [641 37 100 22];
    app.WindowFunctionDropDown.Value = 'Hanning';

    % Create SignalingdurperkeyTdmsEditFieldLabel
    app.SignalingdurperkeyTdmsEditFieldLabel = uilabel(app.UIFigure);
    app.SignalingdurperkeyTdmsEditFieldLabel.HorizontalAlignment = 'right';
    app.SignalingdurperkeyTdmsEditFieldLabel.Position = [927 301 176 22];
    app.SignalingdurperkeyTdmsEditFieldLabel.Text = 'Signaling dur. per key
(Td) (ms)';

    % Create TdReceived
    app.TdReceived = uieditfield(app.UIFigure, 'numeric');
    app.TdReceived.ValueChangedFcn = createCallbackFcn(app,
@TdReceivedValueChanged, true);
    app.TdReceived.Position = [1118 301 100 22];
    app.TdReceived.Value = 250;

    % Create RestingdurationTrmsEditFieldLabel_2
    app.RestingdurationTrmsEditFieldLabel_2 = uilabel(app.UIFigure);
    app.RestingdurationTrmsEditFieldLabel_2.HorizontalAlignment = 'right';
    app.RestingdurationTrmsEditFieldLabel_2.Position = [954 270 149 22];
    app.RestingdurationTrmsEditFieldLabel_2.Text = 'Resting duration (Tr)
(ms): ';

    % Create TrReceived
    app.TrReceived = uieditfield(app.UIFigure, 'numeric');
    app.TrReceived.ValueChangedFcn = createCallbackFcn(app,
@TrReceivedValueChanged, true);
    app.TrReceived.Position = [1118 270 100 22];
    app.TrReceived.Value = 250;

    % Create StartListeningButton
    app.StartListeningButton = uibutton(app.UIFigure, 'push');
    app.StartListeningButton.ButtonPushedFcn = createCallbackFcn(app,
@StartListeningButtonPushed, true);
    app.StartListeningButton.Position = [965 397 100 23];
    app.StartListeningButton.Text = 'Start Listening';

    % Create ReceivedKeys
    app.ReceivedKeys = uieditfield(app.UIFigure, 'text');
    app.ReceivedKeys.FontName = 'Courier New';
    app.ReceivedKeys.FontSize = 18;
    app.ReceivedKeys.FontWeight = 'bold';
    app.ReceivedKeys.Position = [945 430 257 38];

    % Create DecodingAlgorithmDropDownLabel
    app.DecodingAlgorithmDropDownLabel = uilabel(app.UIFigure);
    app.DecodingAlgorithmDropDownLabel.HorizontalAlignment = 'right';
    app.DecodingAlgorithmDropDownLabel.Position = [978 331 109 22];
    app.DecodingAlgorithmDropDownLabel.Text = 'Decoding Algorithm';

```

```

    % Create DecodingAlgorithmDropDown
    app.DecodingAlgorithmDropDown = uidropdown(app.UIFigure);
    app.DecodingAlgorithmDropDown.Items = {'Modulation', 'Filter bank',
'Spectrogram'};
    app.DecodingAlgorithmDropDown.ValueChangedFcn = createCallbackFcn(app,
@DecodingAlgorithmDropDownValueChanged, true);
    app.DecodingAlgorithmDropDown.Position = [1102 331 116 22];
    app.DecodingAlgorithmDropDown.Value = 'Modulation';

    % Create ListenRecordingButton
    app.ListenRecordingButton = uibutton(app.UIFigure, 'push');
    app.ListenRecordingButton.ButtonPushedFcn = createCallbackFcn(app,
@ListenRecordingButtonPushed, true);
    app.ListenRecordingButton.Position = [1084 397 105 23];
    app.ListenRecordingButton.Text = 'Listen Recording';

    % Create PlayGenCheck
    app.PlayGenCheck = uicheckbox(app.UIFigure);
    app.PlayGenCheck.Text = 'Play generated audio while listening';
    app.PlayGenCheck.Position = [968 373 214 22];
    app.PlayGenCheck.Value = true;

    % Create DecodeSuccess
    app.DecodeSuccess = uilabel(app.UIFigure);
    app.DecodeSuccess.FontWeight = 'bold';
    app.DecodeSuccess.Visible = 'off';
    app.DecodeSuccess.Position = [1007 236 195 22];
    app.DecodeSuccess.Text = 'Decoded using modulation';

    % Create ItispossibletochangedecodingalgorithmafterlisteningLabel
    app.ItispossibletochangedecodingalgorithmafterlisteningLabel =
uilabel(app.UIFigure);
    app.ItispossibletochangedecodingalgorithmafterlisteningLabel.FontColor =
[0.502 0.502 0.502];
    app.ItispossibletochangedecodingalgorithmafterlisteningLabel.Position =
[927 192 314 45];
    app.ItispossibletochangedecodingalgorithmafterlisteningLabel.Text = 'It
is possible to change decoding algorithm after listening.';

    % Create Noneedto recordanother timeLabel
    app.Noneedto recordanother timeLabel = uilabel(app.UIFigure);
    app.Noneedto recordanother timeLabel.FontColor = [0.502 0.502 0.502];
    app.Noneedto recordanother timeLabel.Position = [998 181 171 22];
    app.Noneedto recordanother timeLabel.Text = 'No need to record another
time';

    % Show the figure after all components are created
    app.UIFigure.Visible = 'on';

end
end

% App creation and deletion
methods (Access = public)

```

```
% Construct app
function app = termproject24424162442440

    % Create UIFigure and components
    createComponents(app)

    % Register the app with App Designer
    registerApp(app, app.UIFigure)

    % Execute the startup function
    runStartupFcn(app, @startupFcn)

    if nargin == 0
        clear app
    end
end

% Code that executes before app deletion
function delete(app)

    % Delete UIFigure when app is deleted
    delete(app.UIFigure)
end
end
end
```